

Tables of Contents

Chapter 1	8
Introduction	8
1.1. Background.....	8
1.2. Problem Statement.....	10
1.3. Research Objectives and Questions.....	12
1.3.1. Research Objectives	12
1.3.2. Research Questions	12
1.4. Thesis Contribution	13
1.4.1. Academic Significance	13
1.4.2. Methodological Contribution	14
1.4.3. Industrial and Practical Contribution.....	14
1.4.4. Potential for Future Extension.....	15
1.5. Scope and Limitations	15
1.5.1. Scope of the Study.....	15
1.5.2. Limitations.....	16
1.6. Report Structure.....	16
Chapter 2	19
Literature Review	19
2.1. Overview of Aircraft Maintenance and Continuing Airworthiness.....	19
2.2. Maintenance Program and MPD Composition.....	21
2.2.1. Structure and Metadata of MPD Tasks.....	21
2.2.2. From MRBR to MPD to AMP.....	22
2.2.3. Practical Challenges in MPD Data Utilization	22
2.2.4. Role of MPD in Task Packaging Optimization	23
2.3. Maintenance Task Packaging Philosophies and Operational Practice	23
2.3.1. The Rationale for Packaging	24
2.3.2. Block Maintenance Programs.....	24
2.3.3. Equalized (Progressive) Maintenance Programs.....	24
2.3.4. Line and Base Maintenance Context.....	25
2.3.5. Limitations of Traditional Packaging Philosophies.....	25
2.4. Optimization in Aircraft Maintenance	26

2.4.1.	Conceptualization of the Task Packaging Problem	26
2.4.2.	Key Components of Packaging Optimization Model.....	27
2.4.3.	Why Traditional Heuristics are Insufficient	28
2.4.4.	Framing as Optimization Problem.....	28
2.4.5.	Integration of Real-World Constraints	29
2.4.6.	The role of Optimization in Maintenance Engineering	29
2.5.	Basic Theory of the Method (MILP Fundamentals for Maintenance Packaging)....	29
2.5.1.	Fundamentals of Mixed-Integer Linear Programming.....	30
2.5.2.	Why MILP is Suitable for Maintenance Task Packaging.....	30
2.5.3.	MILP Constructs Relevant to Maintenance Packaging.....	31
2.5.4.	Theoretical Advantages of MILP over Heuristic and Metaheuristic Methods..	31
2.5.5.	Enhancements and Scalability Techniques in Modern MILP Applications	32
2.5.6.	Determinism and Reproducibility as Methodological Principles.....	33
2.6.	Review of Prior Research and Identified Gaps	33
2.6.1.	Early Approaches and Heuristic Methods	33
2.6.2.	Metaheuristic and Hybrid Strategies	34
2.6.3.	Deterministic and MILP-Based Studies	34
2.6.4.	Industrial Implementations and Practical Studies	35
2.6.5.	Metaheuristic and Hybrid Strategies	35
2.7.	Economic and Operational Dimensions	36
2.7.1.	Cost Components in Maintenance Task Packaging.....	36
2.7.2.	Early Execution, Fallback and Activation Trade-offs	37
2.7.3.	Workload and Resource Balance	38
2.7.4.	Industrial Cost References and Parameterization.....	38
2.7.5.	Sensitivity and Cost-Function Design	39
2.7.6.	Operational Integration and Practical Benefits.....	39
2.8.	Synthesis and Conceptual Framework	40
2.8.1.	Synthesis of Theoretical and Operational Foundations.....	40
2.8.2.	Conceptual Flow: From Regulation to Optimization	40
2.8.3.	Integration of Cost and Compliance Dimensions.....	41
2.8.4.	Conceptual Framework to the Research.....	41
2.9.	Chapter Summary	42

Chapter 3	46
Methodology.....	46
3.1. Overview of Research Methodology.....	46
3.1.1. Methodological Approach	46
3.1.2. Research Flow and Integration Structure	46
3.1.3. Methodological Scope and Boundaries	47
3.2. Data Framework and Model Inputs.....	47
3.2.1. Data Sources and Structure	48
3.2.2. MPD Data Characteristics and Preprocessing.....	48
3.2.3. Model Parameter Generation.....	49
3.2.4. Variable Clasification	50
3.3. Model Formulation.....	52
3.3.1. Objective Model	53
3.3.2. Model Assumption (Limitation explain in detail)	55
3.3.3. Simplification and Justification for Discrete MILP Design	56
3.3.4. Mathematical Model Summary	57
3.4. Model Implementation	59
3.4.1. Implementation Environment.....	59
3.4.2. Model Translation and Workflow Overview	59
3.4.3. Solver Configuration and Optimization Process	60
3.4.4. Implementation Flow Diagram\	61
3.5. Model Output Design and Interpretation.....	61
3.5.1. Overview of Output Categories.....	62
3.5.2. Task-Package Mapping Matrix.....	63
3.5.3. Cost and Workload Decomposition Outputs	63
3.5.4. Discrete Output Design Rationale	64
3.5.5. Line Maintenance Task Classification.....	65
3.5.6. Output Visualization and Interpretation	65
3.6. Model Validation and Evaluation	66
3.6.1. Quantitative Validation	66
3.6.2. Qualitative Validation	68
3.6.3. Validation Summary and Acceptance Rules.....	68

3.6.4.	Linkage to Research Objective.....	69
3.7.	Summary of Methodology.....	70
3.7.1.	Overview of Methodological Framework	70
3.7.2.	Model Foundation and Variable Definiton	70
3.7.3.	MILP Model Formulation.....	71
3.7.4.	Model Implementation and Output Design	71
3.7.5.	Validation and Evaluation.....	72
3.7.6.	Methodological Coherence and Contribution.....	72
Chapter 4		73
Result and Analysis		73
4.1.	Overview of Experimental Setup and Data Context	73
4.2.	Solver Performance and Feasibility.....	73
4.3.	Optimization Results: Task-Package Assignment	73
4.4.	Cost and Workload Analysis.....	73
4.5.	Comparative Evaluation Against Baseline Method.....	73
4.6.	Sensitivity and Robustness Analysis	73
4.7.	Qualitative Validation and Expert Review	73
4.8.	Discussion and Implications.....	73
4.9.	Summary of Result and Analysis	73
1.	Summary of Key Findings.....	73
2.	Confirmation to Research Objectives.....	73
Chapter 5 Conclusion		74
Reference		75
Appendix A		81
Mathematical Formulation		81
A.1.	Notation Summary (Sets, Indices, Parameters, and Decision Variables)	81
A.1.1.	Sets and indices	81
A.1.2.	Parameters	81
A.1.3.	Precomputed helper sets / indicators	82
A.1.4.	Decision variables.....	82
A.1.5.	Multiplicity / alignment flags (precomputed).....	83
A.2.	Objective Function	83

A.3.	Constraint System and Logical Rules.....	84
A.3.1.	Aggregated assignment from indexed occurrences	84
A.3.2.	Bin time representation.....	84
A.3.3.	Interval Selection.....	84
A.3.4.	Availability Propagation (precomputed indicators).....	84
A.3.5.	Visit Activation and A-onnly linearization	85
A.3.6.	Task Assignment Feasibility.....	85
A.3.7.	Nondecreasing (ordering) of occurrences.....	86
A.3.8.	Rolling Max-Gap (resettable interval) Constraint.....	86
A.3.9.	Early Execution Slack	87
A.3.10.	Line-Forcing Constraint	87
A.3.11.	Workload and Capacity	87
A.4.	Linearization remarks and Big-M choices	88
A.5.	Solver Configuration and Validation Criteria	88
A.5.1.	Feasibility Check.....	88
A.5.2.	Robustness Assessment	89
A.6.	Implementation Mapping	89
Appendix B.....		90
System Architecture.....		90
B.1.	System Architecture Overview	90
B.2.	Data Preprocessing	92
B.2.1.	Purpose and Functional Overview.....	93
B.2.2.	Processing Logic and Flow Description.....	93
B.2.3.	Implementation Script	94
B.2.4.	Flowchart and Data Representation.....	94
B.2.5.	Linkage and Reproducibility	95
B.3.	Baseline Heuristic System.....	96
B.4.	MILP Optimization Implementation	97
B.5.	Execution Environment and Reproducibility	99

Abbreviations

The following abbreviations are used through out the present document :

[illegible]

--	--

Chapter 1

Introduction

1.1. Background

In daily airline operations, MPD tasks are combined into scheduled events—line checks, A-checks, C-checks, and heavier base visits—a process known as **task packaging**. The packaging structure directly determines aircraft downtime, manpower allocation, and maintenance cost. Historically, airlines have used heuristic approaches such as block programs (fixed packages at 400/800/2400 FH intervals) and equalized programs (smoothing workload across events). While practical, these methods frequently cause premature task execution, redundant access, and suboptimal resource use, particularly when dealing with the dense and interdependent task structures of current MPDs [6], [7].

Recent academic studies [6], [8], [9], [10] and industry analyses [11], [12], [13] indicate that **optimization-based formulations**, especially **Mixed-Integer Linear Programming (MILP)**, can substantially reduce total maintenance cost and improve workload balance compared with heuristic grouping. MILP allows complex relationships—interval windows, manpower capacity, access commonality, and cost trade-offs—to be expressed as linear constraints and solved optimally using modern solvers such as **Gurobi**. Peer-reviewed research on maintenance task allocation and packaging shows algorithmic and MILP approaches outperform manual or rule-based methods in controlled experiments [8], [14].

The COVID-19 era highlighted further inefficiencies in static scheduling: task deferrals and fleet-utilization shifts revealed the need for data-driven, adaptable planning tools [12], [13]. These lessons motivate research into optimization models that are both regulatorily compliant and computationally tractable for large-scale MPDs.

Empirical and heuristic approaches have been the dominant industry practice for decades [15]. Airlines typically design check packages manually, aided by spreadsheets or enterprise maintenance systems such as AMOS, RAMCO, or TRAX [20–22]. These systems enforce regulatory compliance but rely heavily on human judgment for packaging logic. Consequently, outcomes are neither globally optimal nor easily reproducible. Several academic studies have attempted to formalize packaging and scheduling logic using:

- **Heuristic algorithms** [6] offer rapid results but depend on parameter tuning and cannot guarantee optimality.

- **Genetic algorithms (GA) and meta-heuristics** [19] show promise in exploring large search spaces; however, their solutions are non-deterministic and auditability may be challenging for regulated environments.
- **Reinforcement learning (RL) frameworks** [16] have been explored for condition-based and dynamic scheduling, but interpretability and certification traceability remain concerns—critical for aviation authorities.

A particularly relevant prior work, **Marvella (2023)** [7], conducted an exhaustive empirical study on the B737-800 NG using operator data from PT XYZ. By enumerating candidate combinations of A- and C-check intervals, Marvella demonstrated measurable reductions in average cost per flight hour. However, the approach was constrained by brute-force enumeration and fixed heuristics, limiting scalability and the ability to incorporate complex constraint interactions such as access overlap or fallback penalties.

In contrast, **MILP formulations** offer a mathematically rigorous alternative. Recent studies by **Ali et al. (2025)** [8], **Korba et al. (2023)** [9], **Witteman et al. (2021)** [14], and **Tseremoglou et al. (2023)** [10] modeled maintenance task allocation, packaging, and fleet scheduling as bin-packing and constraint-satisfaction problems, enabling simultaneous consideration of regulatory, operational, and cost constraints. These models highlight how MILP can represent logical conditions—such as task grouping, exclusivity, and interval compliance—using linear inequalities, producing solutions that are both optimal and auditable [14].

Nonetheless, gaps remain in translating these formulations into airline-ready frameworks. Most published models use synthetic or simplified datasets, omit explicit preprocessing pipelines, and rarely document parameter provenance or sensitivity. Thus, while MILP has strong theoretical potential, its practical integration with real MPD data and cost structures remains under-developed and requires reproducible pipelines and data governance.

This thesis addresses that gap by developing a **constraint-rich, reproducible MILP framework** that integrates regulatory semantics, operational feasibility, and full cost modeling for large MPD datasets. The framework supports auditability, parameter transparency, and scalability, and uses Pyomo [17] and Gurobi [18] together with documented data-preparation steps to enable airline deployment and regulatory review.

1.2. Problem Statement

Although maintenance-program structures have evolved under MSG-3 and MRBR guidance for decades, the operational translation from MPD content to executable check packages remains largely heuristic and manual. Airlines and MROs often rely on spreadsheet-based grouping or enterprise MRO platforms, where optimization capabilities are primarily workflow- and rule-driven; formal optimal task packaging is not typically embedded enterprise MRO platforms. As a result, maintenance events are frequently non-optimal, producing unnecessary early task executions, duplicated access actions, and irregular workload profiles [3], [4], [11]. These problems can be formulated into :

1. **Lack of an integrated optimization model that unifies regulatory, operational, and cost dimensions.**
 - Existing studies often isolate one aspect (e.g., manpower leveling or interval adherence) but do not integrate all simultaneously.
 - No framework currently demonstrates full MSG-3 compliance within an optimization model while maintaining auditable logic suitable for FAA/EASA oversight.
2. **Limited scalability and reproducibility of empirical or heuristic methods.**
 - The exhaustive-search model of Marvella (2023) successfully proved the cost benefit of structured packaging but was computationally infeasible beyond a few hundred tasks and dependent on manual parameter tuning [7].
 - Many heuristic or GA-based studies use simplified datasets and omit reproducibility standards such as unit normalization, metadata provenance, or transparent default assumptions[6], [9].
3. **Incomplete economic modeling of maintenance cost components[11], [12], [13].**
 - Most prior works measure only labor hours or downtime; economic penalties related to early/fallback execution, activation overhead, or workload imbalance are rarely captured in a unified cost function.
 - Industry reports emphasize that indirect factors materially affect total maintenance cost, motivating richer cost structures and re-optimized packaging decisions.
4. **Absence of reproducible data-preprocessing pipelines for real MPDs[6].**

- MPDs contain diverse annotation styles (e.g., NOTE/ACT/CE/TCI), units, and access-zone codes; most optimization studies preprocess these manually [5].
- Without deterministic conversion logic, results are not auditable nor portable between aircraft types.

5. **Need for models compatible with emerging digital and regulatory trends.**

- Recent research explores stochastic or AI-based scheduling, yet regulatory authorities still require transparent deterministic justification for interval compliance [19].
-  ○ As data-driven maintenance expands (CBM, PHM), there remains a critical gap for a baseline optimization layer that produces certified check structures before adaptive scheduling occurs [10], [19].
- Current MRO platforms focus on compliance/workflow, not optimization[20–22]

Mixed-Integer Linear Programming (MILP) provides a mathematically rigorous and regulator-auditable foundation that directly addresses these gaps. It enables:

- Explicit constraint encoding of regulatory, operational, and economic logic;
- Deterministic optimality, producing solutions verifiable under audit conditions;
- Scalability via advanced solver techniques (e.g., tight big-M, warm-starts, valid inequalities); and
- Integrability with digital MRO systems through structured data inputs and outputs.

Recent literature reinforces this choice:

- Algorithmic and MILP formulations outperform heuristic grouping in controlled studies and can manage dense, interdependent task structures [6–10].
- Hybrid approaches improve tractability for large task sets without sacrificing auditability [6].
- Fleet-level optimization demonstrates adaptability to real airline datasets and small-fleet contexts [8].

Therefore, this thesis positions MILP not as a purely mathematical novelty but as a pragmatic bridge between engineering regulation and computational optimization. It transforms an ad-hoc, human-intensive process into a systematic, traceable, and scalable decision framework.

1.3. Research Objectives and Questions

1.3.1. Research Objectives

The overarching aim of this research is to develop and validate an optimization framework for maintenance task packaging that leverages **Mixed-Integer Linear Programming (MILP)** to achieve cost-efficient, regulatorily compliant, and operationally balanced maintenance programs.

This general aim is refined into four specific objectives:

1. **To formulate a constraint-rich MILP model** that represents maintenance-program semantics—interval windows, task-to-check assignment logic, access-zone relationships, manpower limits, and cost trade-offs—derived directly from **Airbus A320 MPD data** [5]. → Links to Chapter 3 §3.2 (Model Framework).
2. **To implement a deterministic preprocessing pipeline** that transforms raw MPD data into solver-ready variables, automatically normalizing units, parsing annotations (NOTE/ACT/CE/TCI), and encoding every assumption for reproducibility and auditability [4], [6]. → Links to Chapter 3 §3.3 (Data Preparation).
3. **To calibrate and validate cost parameters** (labor rate, downtime, activation overhead, and fallback penalties) using open-source industry benchmarks and to perform sensitivity analysis on economic assumptions. → Links to Chapter 4 §4.3 (Sensitivity Testing).
4. **To benchmark MILP performance against heuristic baselines**—block and equalized programs, and Marvella (2023)’s exhaustive search—to quantify improvements in total cost, workload balance, and computational efficiency [7–10]. → Links to Chapter 4 §4.2 (Comparative Results).

Together these objectives ensure that the research not only develops a mathematical model but also delivers a verifiable, transferable, and industry-relevant optimization framework.

1.3.2. Research Questions

Derived from the objectives above, the following questions guide the investigation:

1. **Optimization Logic:**

How can maintenance tasks be assigned to scheduled checks to minimize total economic cost while maintaining full compliance with MPD interval requirements and manpower constraints?

2. **Computational Design:**

Which MILP formulation strategies (e.g., valid inequalities, big-M tightening, warm-start seeding) improve solver tractability and scalability for large-scale MPD datasets?

3. **Reproducibility and Transparency:**

How can preprocessing workflows ensure consistent, traceable translation from OEM MPD data into solver inputs, meeting audit and certification expectations?

4. **Comparative Performance:**

How does the proposed MILP-based framework perform relative to traditional block/equalized and heuristic methods in terms of total cost, workload balance, and runtime efficiency?

1.4. **Thesis Contribution**

1.4.1. **Academic Significance**

This thesis contributes to the growing body of research on optimization-based aircraft maintenance planning by addressing the gap between theoretical MILP formulations and real-world airline implementation. It advances the academic understanding of task-packaging optimization in three key ways:

1. Integration of domain-specific semantics into a mathematical model — The research encodes MSG-3 logic [3], MRBR-derived interval constraints [4], and MPD structure [5] directly into MILP constraint families. — This provides a formal bridge between maintenance regulation and optimization science, filling a gap where most previous studies treated task allocation as a generic bin-packing or scheduling problem.
2. Methodological transparency and reproducibility — A deterministic preprocessing pipeline ensures that the entire modeling process—from MPD ingestion to solver output—is traceable, auditable, and repeatable [4], [6]. — This directly supports open-science and data-governance principles rarely applied to maintenance-optimization studies.
3. Empirical validation on real-world datasets — The research tests its framework using Airbus A320 MPD data [4] calibrated with publicly available industry costs. — This allows comparison with prior empirical baselines such as Marvella (2023)

[7], demonstrating that the MILP model can deliver measurable improvements in total cost per flight hour and workload balance.

Together, these contributions advance both optimization theory and applied maintenance-engineering practice, making the work suitable for academic publication or replication by future researchers.

1.4.2. Methodological Contribution

The thesis introduces a **constraint-rich MILP architecture** that can serve as a reference model for future optimization research in aviation maintenance.

Specific innovations include:

- **Structured variable taxonomy:** clear separation between decision, dependent, control, and outcome variables for model clarity and scalability.
- **Integrated cost modeling:** inclusion of early-execution penalties, fallback costs, and workload imbalance in a unified objective function [6], [8].
- **Deterministic unit normalization:** all FH/FC/calendar conversions handled algorithmically to remove subjectivity [4].
- **Solver enhancement features:** warm-start seeding from heuristic baselines and valid-inequality tightening for faster convergence.

These methodological features make the model extensible for hybrid MILP–metaheuristic or stochastic frameworks in future research [19].

1.4.3. Industrial and Practical Contribution

For industry practitioners—airline engineering departments, MROs, and regulatory authorities—this study provides:

1. A reproducible and auditable optimization tool that supports evidence-based maintenance-program decisions under **FAA/EASA oversight**.
2. Quantified insights into cost trade-offs, showing how labor, downtime, and workload-balancing penalties interact to influence total maintenance cost.
3. A foundation for **digital MRO integration**, enabling future connection with enterprise systems (AMOS, RAMCO, TRAX) for automatic check-package generation.
4. Sensitivity-analysis results that help managers understand which cost parameters (e.g., labor rate, downtime) most affect total program cost.

By formalizing the packaging process, the thesis supports a gradual shift from heuristic or experience-based planning toward data-driven, optimization-guided maintenance management.

1.4.4. Potential for Future Extension

Although the research focuses on scheduled maintenance packaging, the MILP structure can be extended toward:

- **Fleet-level optimization**, integrating multiple aircraft with shared resource constraints [8], [10].
- **Hybrid deterministic–stochastic models** to handle condition-based or opportunistic maintenance [19].
- **Multi-objective optimization**, balancing cost with reliability or sustainability metrics (e.g., carbon impact of downtime).

These directions establish continuity for postgraduate or collaborative research beyond this thesis.

1.5. Scope and Limitations

1.5.1. Scope of the Study

This research focuses on the optimization of scheduled maintenance task packaging for commercial narrow-body aircraft—specifically, the Airbus A320 family—using Mixed-Integer Linear Programming (MILP) as the primary modeling framework. The study centers on transforming MPD task data into cost-optimal, regulatorily compliant maintenance packages. The study scope includes:

1. Scheduled maintenance tasks from the A320 MPD: inspection, lubrication, functional check, and restoration tasks with defined interval values (FH, FC, and calendar).
2. Packaging optimization—assigning each task to a maintenance event (A-, C-, or line-check) under MSG-3-based interval and access constraints.
3. Economic cost modeling that incorporates direct labor, downtime opportunity, activation overhead, and fallback penalties.
4. Data preprocessing and normalization, ensuring unit consistency and provenance for all variables and parameters.

5. Computational implementation using Python (Pyomo) and the Gurobi solver, tested under realistic MPD subsets of 500–1500 tasks.

1.5.2. Limitations

To maintain methodological focus and computational feasibility, the following aspects are deliberately excluded:

1. **Unscheduled or stochastic maintenance tasks** (e.g., corrective actions, condition-based maintenance). These are outside the MSG-3 preventive domain and require probabilistic or predictive models [3], [19].
2. **Multi-aircraft or fleet-level scheduling**. The model optimizes packaging for a single representative aircraft. Extending to multiple aircraft introduces resource allocation and hangar-capacity coupling, reserved for future research [8], [10].
3. **Crew rostering and shift scheduling**. These are handled downstream by maintenance management systems and are not part of packaging optimization [10].
4. **Real-time disruption management**. The MILP framework assumes deterministic input parameters and static planning horizons. Dynamic re-planning under uncertainty will be explored in potential follow-up research [15], [19].
5. **Material logistics and spare-parts availability**. Cost modeling focuses on labor and downtime; material supply-chain factors are assumed constant.

1.6. Report Structure

This thesis is organized into **five core chapters** supported by **technical appendices**. Each chapter builds progressively from conceptual foundations to model implementation, results, and implications. The structure is designed to maintain alignment between the research objectives, methodology, and validation outcomes.

1. Chapter 1 – Introduction

Establishes the regulatory and operational background of aircraft maintenance planning, outlines traditional and optimization-based task-packaging approaches, and identifies existing research gaps. It defines the **objectives, research questions, scope, and significance** of developing a MILP-based optimization framework for maintenance task packaging.

2. Chapter 2 – Literature Review

Presents an in-depth review of the **continuing airworthiness regulatory system** (FAA, EASA), the **MSG-3 methodology**, the **MRB/MRBR process**, and the **composition of the Maintenance Planning Document (MPD)**. It critically evaluates prior research on maintenance packaging, bin-packing optimization, and MILP formulations, highlighting theoretical, computational, and industrial developments from 2000 to 2025. This chapter consolidates the theoretical framework underpinning the model.

3. Chapter 3 – Methodology

Describes the **conceptual and mathematical foundation** of the proposed MILP framework. It includes:

- data-preprocessing design,
- decision-variable and constraint formulation,
- solver configuration (Pyomo + Gurobi),
- validation criteria, and
- evaluation workflow.

Appendices A and B provide detailed **mathematical derivations** and **system-architecture documentation** supporting this methodology.

4. Chapter 4 – Results and Analysis

Presents the application of the MILP model to **Airbus A320 MPD datasets**, including performance comparisons with **block/equalized baselines** and **Marvella (2023)**'s heuristic results. This chapter reports quantitative outcomes (cost, workload balance, solver runtime) and qualitative validation (expert review, regulatory feasibility), followed by sensitivity and robustness analyses.

5. Chapter 5 – Conclusion and Recommendations

Summarizes key findings, methodological contributions, and validation outcomes. It discusses limitations, proposes extensions toward fleet-level and stochastic optimization, and outlines pathways for **industry implementation** and **academic dissemination**.

6. Appendices

- **Appendix A – Mathematical Formulation:** Complete MILP equations, variable definitions, and constraint logic.

- **Appendix B – System Architecture and Implementation:** Python-based pipeline documentation, data-flow diagrams, solver scripts, and reproducibility checklist.

Additional appendices include data provenance tables and solver parameter logs to ensure audit transparency.

Figure 1-2. Thesis Structure Overview (suggested visual)

A simple flow diagram showing:
Problem → **Literature** → **Method** → **Results** → **Conclusion**,
with parallel arrows referencing Appendices A & B for technical depth.

Chapter 2

Literature Review

2.1. Overview of Aircraft Maintenance and Continuing Airworthiness

Aircraft maintenance is a critical component of aviation safety and operational reliability. Aircraft maintenance ensures that every aircraft remains in an airworthy condition throughout its operational life. Both the **Federal Aviation Administration (FAA)** under *14 CFR Part 121 Subpart L* and the **European Union Aviation Safety Agency (EASA)** under *Part-M / Part-CAMO* require each operator to establish and maintain an **Approved Maintenance Program (AMP)** that satisfies continuing-airworthiness obligations [1], [2], [25]. These programs formalize how preventive and inspection tasks must be performed, documented, and approved. These frameworks define the principles of continuing airworthiness—that all aircraft must remain in a condition for safe operation throughout their service life.

At the foundation of every maintenance program is the **MSG-3 (Maintenance Steering Group – 3)** methodology [3]. MSG-3 applies a reliability-centered decision logic that determines the *minimum required preventive tasks* and their intervals—expressed in flight hours (FH), flight cycles (FC), or calendar time—to preserve safety and performance. The output of the MSG-3 process forms the **Maintenance Review Board Report (MRBR)**, which establishes the certified maintenance requirements for a specific aircraft type; EASA’s MRB procedure sets out the acceptable means and governance for this process [4]. Based on the MRBR, the aircraft manufacturer publishes the **Maintenance Planning Document (MPD)** [5]. For modern fleets such as the Airbus A320 family, the MPD may include thousands of individual tasks with varying interval types—measured in flight hours, flight cycles, or calendar limits—making manual grouping of tasks into efficient maintenance events increasingly impractical [5].

Because the MPD defines the *minimum content* of scheduled maintenance, operators must transform it into a workable execution plan. Tasks that share similar intervals or access requirements are combined into **maintenance packages**—commonly referred to as **A-checks**, **C-checks**, or other visit types[11]. This process, called **maintenance task packaging**, is central to balancing compliance, downtime, and resource efficiency. While the regulatory framework dictates *what* must be done and *when* (interval limits), the packaging logic determines *how* those tasks are grouped into physical maintenance events[6], [8], [14].

Historically, airlines have implemented this grouping through two primary philosophies:

- **Block Programs**, where tasks are bundled at fixed cumulative intervals (for example A-checks every 500 FH and C-checks every 5 000 FH). Although straightforward, block packaging often leads to premature task execution and under-utilization of allowable intervals[11].
- **Equalized or Progressive Programs**, which distribute tasks more evenly across visits to smooth workload and reduce downtime peaks. However, equalized systems are complex to manage manually and can still produce non-optimal groupings when inter-task dependencies or access overlaps are ignored[23], [28].

Under both approaches, the ultimate goal remains identical: to minimize aircraft downtime and maintenance cost while ensuring every MPD task is executed within its approved interval limits. Yet, as the number of tasks and interval combinations in an MPD grows, manual or heuristic packaging becomes infeasible. For example, the A320 MPD includes thousands of items with mixed interval types (FH/FC/calendar) and interrelated access requirements[5], [23]. Consequently, **optimization of maintenance task packaging** emerges as a natural research and engineering challenge. The task-packaging problem sits at the intersection of regulatory compliance, operational practicality, and economic efficiency. From a modeling viewpoint, it can be framed as an **assignment problem**, where each MPD task (i) must be assigned to one or more maintenance events (j) such that:

- interval constraints (MRB/MPD limits) are respected;
- manpower and access constraints are satisfied; and
- total economic cost—including labor, downtime, and early/fallback penalties—is minimized.

Because these relationships involve binary task-to-event decisions and linear cost structures, **Mixed-Integer Linear Programming (MILP)** provides an appropriate mathematical foundation for representing the packaging logic [6], [8], [14], [24]. MILP enables the encoding of logical conditions such as “*task A must occur in the same package as task B if both share an access zone*” or “*task C cannot occur earlier than its threshold interval*” through linear constraints. The resulting formulation is deterministic, auditable, and fully compatible with regulatory oversight—qualities essential in aviation maintenance environments.

In summary, continuing-airworthiness regulation defines *why* maintenance must occur, MSG-3 and the MRBR define *what* tasks must be done, and the MPD provides *the raw data* from which packages are constructed. Optimization of maintenance task packaging defines *how*

those tasks are efficiently grouped into compliant and cost-effective packages. Understanding this regulatory-to-operational chain establishes the context for all subsequent theoretical and methodological discussions in this thesis.

2.2. Maintenance Program and MPD Composition

The **Maintenance Planning Document (MPD)** serves as the principal data foundation for maintenance task packaging. It consolidates all approved maintenance requirements that originate from the **Maintenance Review Board Report (MRBR)** and additional manufacturer or regulatory sources such as **Airworthiness Limitations (ALS/AWL)**, **Certification Maintenance Requirements (CMR)**, **Service Bulletins (SBs)**, and **Airworthiness Directives (ADs)** [1–4]. For each aircraft type, the MPD functions as the technical reference that defines every maintenance task, its interval criteria, its method of accomplishment, and its relationship to other maintenance documentation.

2.2.1. Structure and Metadata of MPD Tasks

An MPD typically contains **thousands of individual maintenance tasks**, each represented as a structured record comprising several key metadata fields[5], [23], [28]:

- **Task Identification** – A unique MPD reference number that links the task to the applicable system, zone, or structure.
- **Interval Data** – The specified maximum thresholds and repeat intervals expressed in **flight hours (FH)**, **flight cycles (FC)**, and/or **calendar days**.
- **Access Requirement** – The physical access or panel opening required, which determines co-locatable tasks.
- **Skill Code** – Maintenance discipline such as Airframe (AF), Avionics (AV), or Powerplant (PP).
- **Duration and Man-hours (MH)** – Estimated workload for planning and resource balancing.
- **Applicability / Effectivity** – The aircraft serial range or modification status to which the task applies.
- **Notes or Special Instructions** – Execution dependencies, required tools, or environmental conditions.

This metadata structure is crucial in the **optimization of maintenance task packaging**, because each field corresponds to one or more model parameters. For example, interval data

define the allowable packaging window, access requirements imply grouping constraints, and man-hour values feed directly into package-level capacity constraints.

2.2.2. From MRBR to MPD to AMP

The MPD does not itself dictate when or how tasks are combined; it provides only the regulatory minima. Operators are required to translate the MPD into their **Approved Maintenance Program (AMP)**, which is tailored to their operational environment, fleet utilization rate, and reliability performance. This transformation process can be summarized as follows[3], [4], [25]:

MRBR (regulatory minima) → MPD (OEM technical baseline) → AMP (operator-specific implementation)

During AMP development, planners analyze each MPD task's interval, access, and effectivity to determine potential grouping. **Packaging** is therefore the practical step between MPD and AMP execution. The quality of the packaging directly influences AMP efficiency—poorly chosen groupings may lead to early task performance, redundant access, or unbalanced manpower loads.

2.2.3. Practical Challenges in MPD Data Utilization

Although the MPD provides detailed technical data, it presents several **data-preparation challenges** when used for optimization modeling[28]:

1. **Heterogeneous interval units.** Tasks can have FH, FC, or calendar intervals, requiring normalization into a unified time base or horizon before optimization.
2. **Threshold ambiguity.** Certain tasks include initial thresholds different from their repeat intervals, which complicates consistency checks when determining first-occurrence bins.
3. **Annotation parsing.** MPDs often use shorthand like *NOTE*, *ACT*, or *CE* (Check Event) that must be interpreted programmatically.
4. **Access overlap.** Tasks sharing a common panel or zone should ideally be grouped to avoid repeated opening and closing actions, yet such relationships are not explicitly encoded.

5. **Effectivity and modification status.** Tasks may only apply to certain aircraft within a fleet; a packaging model must filter tasks according to fleet configuration before optimization.
6. **Incomplete cost attributes.** While man-hours are often available, direct material cost or line-activation cost usually must be estimated from operator databases or industry benchmarks.

These issues mean that **data preprocessing** becomes a necessary bridge between raw MPD content and any optimization engine. The packaging optimization model relies on well-structured input data where each task's attributes are numerically and logically consistent.

2.2.4. Role of MPD in Task Packaging Optimization

Within the context of this thesis, the MPD serves as the **primary input layer** for the Mixed-Integer Linear Programming (MILP) model that optimizes task packaging. Each MPD task is treated as an atomic unit of work characterized by its interval, duration, and associated attributes. The MILP formulation later described in Chapter 3 transforms these attributes into decision variables and constraints that determine:

- which package (or maintenance visit) each task belongs to;
- whether two tasks with shared access or zone identifiers should be combined;
- and how workload and cost are balanced across all packages within the planning horizon.

Because the MPD originates from regulated sources (MRBR, ALS, CMR, etc.), its use ensures that the resulting optimization remains **traceable and auditable** under continuing-airworthiness standards. Thus, the MPD is not only a data source but also a regulatory boundary that defines the feasible space of any maintenance-packaging solution.

2.3. Maintenance Task Packaging Philosophies and Operational Practice

Aircraft maintenance is governed by the principle of **interval-based compliance**: each maintenance task must be performed within its approved limit, defined by flight hours (FH), flight cycles (FC), or calendar time. However, performing each task independently would be operationally and economically infeasible. Airlines therefore group tasks into **packages**, also known as **checks** or **visits**, which are executed together when the aircraft is removed from service. The logic governing how tasks are grouped is known as **maintenance task packaging philosophy**.

2.3.1. The Rationale for Packaging

Packaging consolidates tasks that share compatible intervals or access requirements so that multiple inspections or replacements can be done concurrently. This minimizes repeated disassembly and reassembly, reduces aircraft downtime, and optimizes manpower utilization. A well-designed package aligns with both regulatory compliance and operational economics: all tasks remain within their limits, and total downtime over a maintenance cycle is minimized.

Each package can therefore be viewed as a *logical container* of tasks. For a given MPD with thousands of tasks, determining which task belongs in which package becomes a **combinatorial optimization** problem. This problem grows rapidly in complexity as interval combinations, access overlaps, and cost interdependencies multiply[5], [11], [14]. The following philosophies historically emerged to manage this complexity manually or heuristically.

2.3.2. Block Maintenance Programs

The **block program** (or *letter check* system) is the traditional maintenance philosophy that dominated airline operations for decades. Under this approach, tasks are grouped into predefined checks based on fixed multiples of operating time. For instance, the **A-check** may occur every 500 flight hours and the **C-check** every 6,000 flight hours. Tasks whose intervals are shorter than or equal to the check interval are bundled into the corresponding block[11], [23].

While block programs are simple and easy to audit, they lead to **early task execution**. For example, a task due every 1,000 FH will be performed twice within 2,000 FH if the block interval is 500 FH, even though its actual requirement would permit one execution. This premature execution consumes labor and spares unnecessarily and shortens component on-wing time. Moreover, as aircraft utilization patterns evolve, rigid block intervals fail to adapt to changing operational conditions[25].

Despite these inefficiencies, block programs persist in many airlines because they are easy to plan, document, and communicate to authorities. Their deterministic nature aligns well with regulatory oversight, but they offer limited flexibility and cost efficiency.

2.3.3. Equalized (Progressive) Maintenance Programs

In contrast, **equalized** or **progressive maintenance programs** seek to distribute tasks more evenly across a series of smaller visits, avoiding large downtime peaks. Rather than bundling all tasks at fixed milestones, tasks are phased so that workload and access actions are

balanced across the maintenance cycle [23], [28]. This allows the aircraft to remain operationally available for longer stretches and reduces the magnitude of individual check events.

However, equalized programs are **computationally and logistically complex** to design. Tasks with incompatible intervals or access zones may be forced into suboptimal groupings. Manual planners rely heavily on spreadsheets and expert judgment to find feasible combinations, often leading to inconsistencies or compliance risks. Additionally, equalization can inadvertently create multiple partial accesses to the same zone at different times, eroding some of the expected efficiency gains[25].

The complexity of balancing thousands of interval-based tasks with multiple interdependencies—while ensuring each task remains within its tolerance window—renders purely heuristic or manual approaches inadequate. This limitation motivates the use of formal optimization techniques.

2.3.4. Line and Base Maintenance Context

Task packaging must also respect the organizational and physical boundaries between **line maintenance** and **base maintenance**.

- **Line maintenance** covers frequent, low-duration tasks such as transit checks and A-checks, typically performed overnight or between flights.
- **Base maintenance** involves deeper inspections or structural work (e.g., C-checks, structural inspections), requiring hangar space and extended downtime.

The packaging problem must ensure that tasks assigned to each package are **compatible with available maintenance resources and locations**. While this constraint is usually treated operationally rather than mathematically, it influences packaging feasibility—certain access actions or structural inspections can only be performed under base-maintenance conditions.

The distinction between line and base maintenance is thus an implicit boundary condition in any packaging optimization model: each task's attributes (duration, access zone, equipment need) indirectly determine which maintenance environment it can belong to. This ensures the resulting packages are not only compliant and efficient but also operationally executable.

2.3.5. Limitations of Traditional Packaging Philosophies

Both block and equalized programs rely on **heuristic or experience-based decision-making**, which has several limitations when viewed from an optimization standpoint[11], [14], [23]:

1. **Non-optimal resource utilization.** Early or late task execution leads to uneven workloads and excess cost.
2. **Lack of reproducibility.** Packaging decisions often depend on individual planner experience, making consistency difficult across fleets or operators.
3. **Inflexibility to operational change.** Changes in utilization rate or maintenance capacity require manual restructuring of entire programs.
4. **Hidden dependencies.** Access and task interrelations are often handled implicitly, resulting in redundant or conflicting groupings.
5. **Limited quantification of cost trade-offs.** Traditional methods cannot systematically evaluate how adjusting task intervals or groupings affects total maintenance cost.

Given these limitations, **optimization of maintenance task packaging** using quantitative models—particularly **Mixed-Integer Linear Programming (MILP)**—offers a deterministic and auditable alternative. MILP allows planners to explore all feasible combinations of task groupings within regulatory and operational constraints, guaranteeing that the resulting solution is either optimal or near-optimal under defined objective functions[17], [18], [25]. This transition from heuristic judgment to mathematical optimization represents a paradigm shift in how maintenance programs can be engineered and justified to authorities.

2.4. Optimization in Aircraft Maintenance

The process of maintenance task packaging can be viewed as a **combinatorial optimization problem** whose goal is to determine the most efficient grouping of maintenance tasks into packages (or check events) while satisfying technical, regulatory, and operational constraints. This section frames the problem conceptually and highlights how it aligns with optimization theory, setting the foundation for the Mixed-Integer Linear Programming (MILP) formulation presented later in Chapter 3.

2.4.1. Conceptualization of the Task Packaging Problem

Each task defined in the Maintenance Planning Document (MPD) has three essential interval parameters—**flight hours (FH)**, **flight cycles (FC)**, and **calendar time**—along with additional attributes such as access zone, skill requirement, and man-hours[5], [28]. When tasks share similar intervals or access requirements, they can be executed together within a single maintenance event. Determining the optimal set of such groupings is non-trivial because:

- The number of possible combinations increases exponentially with the number of tasks.
- Each combination affects the total cost, downtime, and future task feasibility.
- Packaging decisions are **interdependent**: placing one task early or late alters the feasibility of others.

Thus, the packaging problem is not merely a matter of scheduling events on a calendar but of finding **combinatorial consistency** among thousands of interval-constrained tasks. From an operations-research perspective, this problem is analogous to a **generalized bin-packing** or **set-partitioning** problem[14], [24]:

- Each *bin* (or package) represents a maintenance visit.
- Each *item* (or MPD task) must be placed in one or more bins such that its interval constraints are satisfied.
- The *objective* is to minimize total cost, which includes labor, downtime, and penalties for early or overlapping execution.

Because tasks and packages involve discrete decisions—whether a task is included in a given package or not—the problem naturally fits the domain of **discrete optimization**.

2.4.2. Key Components of Packaging Optimization Model

Before formal mathematical modeling, it is useful to define conceptually the building blocks of the optimization framework:

1. **Decision Variables** – Represent whether a specific task is assigned to a particular maintenance package.
2. **Constraints** – Express regulatory and operational rules, such as:
 - Interval compliance (tasks must not exceed MRB/MPD limits).
 - Access compatibility (tasks sharing access zones should coincide).
 - Capacity limits (packages cannot exceed manpower or duration thresholds).
 - Exclusivity (each task can be assigned only once within the defined horizon).
3. **Objective Function** – Quantifies the overall efficiency of the packaging arrangement, typically minimizing total cost or maximizing resource utilization.

These components define the logical structure of a packaging problem independent of any specific mathematical solver[17], [24].

2.4.3. Why Traditional Heuristics are Insufficient

Airlines historically addressed task packaging through rule-based or manual methods—assigning tasks to A-, C-, or structural checks based on experience, standard intervals, or spreadsheet calculations[11], [23]. Such heuristic approaches provide feasible but rarely optimal solutions because they:

- Evaluate limited subsets of combinations rather than the full feasible space.
- Lack quantitative consistency; two planners might generate different packaging plans for identical MPD data.
- Do not measure trade-offs between early execution costs and manpower balance.

As modern aircraft such as the A320neo and B737 MAX families carry MPDs containing more than 3 000 tasks, manual grouping cannot guarantee compliance or cost optimality. **Optimization models** provide the only systematic means to evaluate every feasible grouping under all constraints simultaneously.

2.4.4. Framing as Optimization Problem

Given its binary assignment nature and linear cost relationships, the maintenance task packaging problem can be represented in the following generic optimization structure [6][8][14][24]:

- **Decision Space:** Task-to-package assignment matrix.
- **Feasible Region:** All combinations that satisfy interval, access, and capacity constraints.
- **Objective:** Minimize total cost while maintaining full compliance.

This structure belongs to the class of **Mixed-Integer Linear Programs (MILPs)**, where continuous variables (e.g., cumulative cost or workload) coexist with binary variables (task inclusion decisions). MILP offers several advantages for maintenance applications [6][8][14][24]:

- Deterministic and reproducible results—critical for regulatory auditability.
- Ability to encode multiple constraint types within a single unified framework.
- Direct interpretability of outputs (which task belongs to which package and why).
- Compatibility with modern solvers (e.g., Gurobi, CPLEX) that can handle large-scale industrial instances efficiently.

2.4.5. Integration of Real-World Constraints

While the conceptual structure above mirrors classical combinatorial optimization, aircraft maintenance packaging introduces unique domain-specific constraints:

- **Interval Tolerances and Thresholds:** Tasks may have initial thresholds different from repeats, requiring dynamic packaging boundaries[5].
- **Access Dependencies:** Tasks sharing panel or zone identifiers should ideally be performed together to avoid repeated dismantling[28]
- **Workload Balance:** Packages must remain within manpower and duration limits dictated by maintenance base capability.
- **Check Families:** Packages are grouped hierarchically (e.g., A-check families A1–A8 forming one cycle) that define repeating patterns within the planning horizon[23].

These characteristics give the packaging problem a highly structured yet constrained search space—ideal for MILP, which can systematically explore feasible solutions under such mixed discrete-continuous logic[24].

2.4.6. The role of Optimization in Maintenance Engineering

Applying optimization to maintenance task packaging serves multiple strategic goals:

1. **Regulatory Assurance:** Optimization ensures every MPD task is executed within limits and provides a traceable audit trail of packaging decisions.
2. **Cost Efficiency:** Quantitative evaluation of early-execution penalties and downtime costs produces financially optimized packages.
3. **Operational Predictability:** Deterministic models reduce reliance on planner intuition and enable reproducibility across fleets and operators.
4. **Program Scalability:** Mathematical models can be re-run when new tasks or interval changes are introduced, ensuring the AMP remains dynamically optimized.

Through these capabilities, optimization transforms packaging from an experience-driven process into an evidence-based engineering design task.

2.5. Basic Theory of the Method (MILP Fundamentals for Maintenance Packaging)

Optimization of maintenance task packaging involves deciding how thousands of MPD-defined tasks should be grouped into maintenance packages while minimizing overall maintenance cost and ensuring compliance with regulatory limits. The decision logic

underlying this problem is inherently discrete: each task must either be included or excluded from a given package. This discrete, rule-constrained structure forms the core reason **Mixed-Integer Linear Programming (MILP)** is an appropriate theoretical framework for modeling and solving the packaging problem.

2.5.1. Fundamentals of Mixed-Integer Linear Programming

A **Mixed-Integer Linear Program (MILP)** is an optimization model that minimizes or maximizes a linear objective function subject to linear constraints, in which some of the decision variables are restricted to integer or binary values [17][24]. Formally, a MILP can be expressed in its general form as:

$$\text{Minimize } c^T x, \quad \text{subject to } Ax \leq b, \quad x_j \in \mathbb{Z} \text{ for } j \in I, \quad x_j \geq 0 \quad (1)$$

where $c^T x$ represents the cost function, A and b define the constraint system, and some elements of x are integers or binaries[24].

This structure allows MILP to encode logical “yes/no” or “either/or” decisions through binary variables, while continuous variables handle cumulative quantities such as total cost, time, or workload. In maintenance task packaging, this framework maps naturally onto the decision space[6][8][14][24]:

- **Binary variables** represent whether a task is assigned to a particular package.
- **Continuous variables** can represent aggregate cost or total man-hours within a package.
- **Linear constraints** capture regulatory and operational relationships among tasks and packages.

2.5.2. Why MILP is Suitable for Maintenance Task Packaging

The fundamental characteristics of MILP directly align with the nature of the task-packaging problem[6][8][14][24]:

1. **Discrete decision logic** — Each task i must belong to one or more packages j ; such yes/no assignment decisions are inherently binary.
2. **Linear regulatory and resource relationships** — Interval tolerances, workload capacities, and cost formulations can be expressed linearly.
3. **Multiple constraint categories** — MILP can simultaneously accommodate interval limits, access dependencies, skill-capacity constraints, and early/fallback penalties.

4. **Optimality and reproducibility** — MILP guarantees global optimality (or a provable optimality gap) within defined constraints, making it transparent and auditable for regulatory justification.
5. **Scalability** — Modern MILP solvers can efficiently handle thousands of tasks and constraints using advanced branch-and-cut algorithms.

For these reasons, MILP offers a **deterministic and regulator-compatible** way to derive packaging configurations that are cost-effective and technically feasible — a critical advantage over heuristic or stochastic approaches.

2.5.3. MILP Constructs Relevant to Maintenance Packaging

While MILP is a general optimization framework, specific modeling patterns are particularly relevant to the maintenance packaging context:

- **Binary assignment constraints:** Ensure that each task is assigned to exactly one package within the planning horizon.
- **Interval-window constraints:** Maintain compliance with MRB or MPD-defined thresholds and intervals, preventing early or late execution.
- **Access-coupling constraints:** Enforce that tasks sharing an access zone are grouped in the same package.
- **Capacity constraints:** Limit total man-hours or duration per package to reflect maintenance-base resource capacity.
- **Cost linearization:** Represent total maintenance cost as a linear function of workload, downtime, and early-execution penalties[13].

These constructs together create a mathematical structure that reflects how maintenance planners think operationally but allows rigorous optimization through solver-based exploration of feasible solutions.

2.5.4. Theoretical Advantages of MILP over Heuristic and Metaheuristic Methods

MILP provides distinct theoretical and practical advantages over heuristic and metaheuristic techniques (e.g., greedy algorithms, genetic algorithms, or simulated annealing):

Criterion	Heuristic / Metaheuristic	MILP
Optimality	Produces approximate or local optima [19][31][32][34]	Produces globally optimal or provably near-optimal solution [6][8][14][24]

Reproducibility	Stochastic; results may vary between runs [19][31]	Deterministic; identical input yields identical output.
Constraint handling	Difficult to enforce strict interval or logic constraints [19]	Handles complex linear and logical constraints exactly [17]
Auditability	Often opaque or heuristic-based [19][31]	Fully transparent and traceable — essential for regulatory compliance [24]
Scalability	Fast for small problems, unreliable for large datasets [19][31][32]	Modern solvers scale efficiently with preprocessing and valid inequalities [18]

Given the strict compliance environment of aviation maintenance, **reproducibility and auditability** are as important as numerical optimality. MILP’s ability to guarantee both makes it uniquely aligned with airworthiness program requirements.

2.5.5. Enhancements and Scalability Techniques in Modern MILP Applications

Although MILP is theoretically NP-hard [24], modern computational advancements have significantly improved its practicality for large-scale industrial problems [17][18]. Common techniques that enhance MILP performance for maintenance packaging include [6][8][14]:

- **Tight Big-M formulation** – Reduces solver search space by carefully bounding binary–continuous relationships, improving convergence.
- **Valid inequalities and cuts** – Introduce additional logical constraints that prune infeasible or redundant regions of the search tree [24].
- **Warm-start strategies** – Use feasible solutions from previous runs (e.g., block or equalized programs) to accelerate solver performance [7][15].
- **Problem decomposition** – Divide large MPDs into sub-fleets or check families (A1–A8) to maintain tractable problem sizes [6][8].
- **Preprocessing filters** – Remove tasks with unique intervals or fixed dependencies prior to optimization to simplify the model [28].

These techniques ensure that MILP-based optimization remains computationally feasible even when applied to large MPD datasets involving thousands of tasks and constraints.

2.5.6. Determinism and Reproducibility as Methodological Principles

An often-overlooked strength of MILP is its **determinism**—given identical input data and solver settings, it will always produce the same result. This characteristic is crucial in aviation, where repeatability of engineering justification is a regulatory requirement. Optimization results can be validated, documented, and re-audited, providing a defensible link between data, model, and final AMP packages. Furthermore, the preprocessing and data-mapping steps (detailed later in Chapter 3) ensure that all regulatory logic from the MPD is preserved throughout the optimization process, maintaining full traceability from certified intervals to optimized packaging outcomes.

2.6. Review of Prior Research and Identified Gaps

Optimization of **maintenance task packaging** has gradually evolved from manual engineering heuristics into a structured research field combining operations research, data analytics, and maintenance engineering [3][6][8][14]. While numerous studies have explored related topics such as aircraft maintenance scheduling, fleet maintenance planning, and manpower optimization [11][15][16], only a limited subset directly addresses the **task packaging problem**—the efficient grouping of MPD-defined tasks into maintenance packages under regulatory and resource constraints [4][5][23][28]. This section reviews key contributions in that area and identifies remaining research gaps.

2.6.1. Early Approaches and Heuristic Methods

Initial efforts to improve maintenance packaging were largely heuristic. Airlines traditionally relied on **rule-based grouping** guided by planners' experience, Excel macros, or simple decision rules (e.g., grouping by equal or multiple intervals) [11][23].

- These methods achieved operational feasibility but lacked reproducibility and theoretical rigor.
- They could not explicitly balance cost or workload and often led to inconsistent packaging across similar fleets.

The **block and equalized philosophies** (Section 2.3) represent the culmination of these heuristics—simple to implement but unable to quantify trade-offs or systematically handle access dependencies.  Industry references such as *Aircraft Commerce* (2023) and *Airbus MPD guidance* (2024) acknowledge that manual or spreadsheet-based packaging remains prevalent, particularly in smaller operators, due to its simplicity despite known inefficiencies.

2.6.2. Metaheuristic and Hybrid Strategies

With increasing computational power, researchers began applying metaheuristic algorithms—notably Genetic Algorithms (GA), Simulated Annealing (SA), and hybrid MILP approaches—to improve upon manual methods [19][31][32][34]. Studies such as Ali et al. (2025) proposed MILP-based task allocation for aircraft maintenance [8], and other works explored GA, SA, and hybrids to reduce total maintenance cost and improve interval utilization [19][31][32][34]. While these methods achieved partial success in smaller datasets, several limitations remain:

- Metaheuristics offer **no guarantee of global optimality**, producing results that can vary between runs[19][31][32][34].
- Constraint integration is often simplified—interval limits, access overlaps, and labor constraints are modeled only approximately[19][31][32].
- Solutions are **non-reproducible**, limiting regulatory acceptance where deterministic justification is required.

Despite these drawbacks, these studies demonstrated the feasibility of applying quantitative algorithms to packaging problems and helped identify the key constraint relationships later formalized in MILP models.

2.6.3. Deterministic and MILP-Based Studies

The shift toward deterministic optimization, especially **Mixed-Integer Linear Programming (MILP)**, emerged in the early 2020s as researchers sought methods that were both auditable and mathematically exact. Key contributions include:

- **Witteman (2021)**, who formulated a MILP model for maintenance task grouping using binary task-check variables, capturing cost, interval compliance, and workload balance [14].
- **Muchiri (2023)**, who applied a heuristic enumeration combined with partial MILP formulation for the Boeing 737-800 NG, producing one of the first applied examples of packaging optimization in a commercial setting [15].
- **Filho et al. (2024)**, who proposed an innovative method to solve maintenance task allocation and packing [6].
- **Tseremoglou et al. (2023)**, who extended MILP logic to component maintenance and multi-aircraft contexts, introducing resource-pool and base-capacity constraints [10].

- **Korba et al. (2023)**, who presented algorithmic approaches on real-life data [9].

These studies confirm MILP's suitability for task packaging because it allows direct encoding of interval-compliance, workload, and access-synergy constraints [6][8][14][24]. However, most models were built for limited datasets and did not fully address preprocessing, scalability, or integration with MPD structures. The present research aims to overcome these limitations through systematic data handling, scalability enhancements, and formal cost modeling.

2.6.4. Industrial Implementations and Practical Studies

In practice, commercial MRO software such as AMOS, TRAX, and RAMCO includes limited task-grouping or check-building functions, serving as rule-based schedulers that assist human planners rather than true optimization engines [20][21][22]. Industry reports indicate that while digital maintenance planning has improved data accessibility, most systems still rely on manual logic for packaging decisions [11][12][23]. Airbus MPDs continue to serve as static references; only a few operators employ algorithmic optimization, typically in fleet-reliability or interval adjustments rather than in packaging design itself. This gap underscores the need to bridge academic modeling and operational implementation through practical, data-driven, regulator-aligned optimization models.

2.6.5. Metaheuristic and Hybrid Strategies

Despite progress in deterministic and heuristic modeling, several open challenges persist in the context of **maintenance task packaging optimization**:

1. **Integration of full MPD data structures** – Most models rely on simplified task lists without processing real MPD metadata (e.g., NOTE/ACT/CE annotations, mixed interval units).
2. **Scalability to large problem instances** – Few studies address computational efficiency for full-fleet datasets exceeding 1 000 tasks, typical for narrow-body aircraft.
3. **Unified cost modeling** – Existing approaches often focus solely on labor hours or access frequency, neglecting activation cost, downtime penalties, and indirect effects.
4. **Preprocessing and data normalization** – Limited research exists on transforming MPD data into solver-ready, normalized parameters while preserving regulatory traceability.

5. **Regulatory compliance and auditability** – Heuristic and metaheuristic solutions remain difficult to justify to authorities; deterministic MILP offers a solution but requires structured validation frameworks.
6. **Digital integration and reproducibility** – Few studies incorporate reproducibility features or deterministic preprocessing pipelines consistent with continuing-airworthiness standards.

Addressing these gaps forms the rationale for the current thesis. By combining MILP's mathematical rigor with structured MPD preprocessing and realistic cost representation, this research advances maintenance task packaging optimization toward practical, auditable application in airline environments.

2.7. Economic and Operational Dimensions

The design of maintenance task packages is not only a technical compliance problem but also a significant **economic optimization challenge**. Each packaging decision influences direct and indirect maintenance costs, aircraft downtime, manpower distribution, and operational continuity [13]. Therefore, understanding the cost and operational dimensions of task packaging is essential for developing a meaningful optimization model [6][8][14].

2.7.1. Cost Components in Maintenance Task Packaging

Aircraft maintenance cost can be broadly categorized into **direct**, **indirect**, and **opportunity** costs. Each category interacts differently with task packaging decisions:

1. Direct Costs

These are costs directly associated with performing maintenance work:

- **Labor cost** — The total man-hours multiplied by labor rates, often differentiated by skill categories (airframe, avionics, powerplant) [13].
- **Material and consumable cost** — Spare parts, lubricants, or inspection kits [11].
- **Access cost** — Time and labor required to open and close panels or remove components for access, which can be minimized through task grouping [5][28].

2. Indirect Costs

Indirect costs stem from resource utilization and support functions:

- Facility occupancy (hangar space, ground equipment) [11].

- **Activation cost** (Administrative overhead) — Fixed cost associated with initiating a maintenance event (e.g., docking, hangar setup, tooling preparation) [13].
- Tooling amortization and calibration cycles [11].

These costs are often proportional to total maintenance duration and can be reduced by minimizing the number of separate maintenance events [13].

3. **Opportunity Costs**

The largest hidden cost driver in packaging decisions is **aircraft downtime**. Every maintenance event removes the aircraft from service, reducing potential revenue generation. Poorly optimized packaging — for example, premature task execution or excessive check frequency — increases downtime and decreases operational efficiency.

In the context of optimization, these cost components can be represented linearly and integrated into an objective function that balances **execution cost** and **downtime cost**, forming the foundation of a total-cost-minimization problem.

2.7.2. **Early Execution, Fallback and Activation Trade-offs**

A defining characteristic of task packaging is the **trade-off between early execution and deferred (fallback) execution**.

- **Early execution** occurs when a task is performed before its interval threshold to align with other tasks in the same package. While it reduces the number of maintenance visits, it wastes potential remaining life on components and increases cumulative labor cost [11].
- **Fallback execution** occurs when a task is delayed close to or beyond its interval limit. This may violate compliance if not properly controlled but can sometimes reduce total downtime if properly managed within tolerance limits.

Optimization must therefore find the balance between these two forces, guided by economic penalties. MILP models can represent this relationship through cost coefficients assigned to early and late execution terms, ensuring that the solver balances package compactness against interval utilization [6][14][24].

The **activation cost**, representing the fixed cost of initiating a package, further reinforces this trade-off: fewer, larger packages reduce activation costs but increase early-task penalties; more, smaller packages reduce early-task penalties but increase activation frequency. This **cost-**

interaction dynamic is the central economic driver of the packaging optimization problem [6][14].

2.7.3. Workload and Resource Balance

From an operational standpoint, a feasible packaging solution must respect **maintenance-base capacity** — manpower, work-shifts, and duration limits. Uneven distribution of tasks among packages can overload certain maintenance visits while underutilizing others. Balanced workload is therefore both an operational necessity and an implicit cost consideration [11].

MILP-based packaging models account for workload balance by enforcing upper limits on total man-hours or task durations per package [6][8][14]. The objective function can also include a **workload variance penalty**, indirectly promoting smoother distribution of labor across all packages. This improves predictability for manpower planning and material procurement, reducing operational bottlenecks [13].

2.7.4. Industrial Cost References and Parameterization

In real-world implementation, accurate cost parameterization is vital for meaningful optimization. Airlines and MROs typically use a combination of internal data and external benchmarks to define the cost parameters used in packaging models:

- **Labor rate (r_{lab})** — Derived from internal maintenance cost accounting (USD per man-hour).
- **Downtime cost (r_{down})** — Based on aircraft lease or revenue-equivalent rates (USD per hour of ground time) [13].
- **Activation overhead (ω)** — Estimated per check event, including hangar setup and logistics [11].
- **Access cost (include in MH_i)** — Based on typical time/labor for panel opening, often derived from task cards or MPD access categories [5][28].

Industry sources such as *Aircraft Commerce (2023)*, *Naveo (2024)*, and OEM cost estimation guides provide empirical benchmarks for calibrating these parameters [11][12][23]. For narrow-body aircraft such as the Airbus A320, activation and downtime costs typically dominate the optimization trade-offs, while labor costs scale more predictably with workload. These empirical foundations ensure that optimization outputs reflect realistic operational economics rather than abstract mathematical efficiency.

2.7.5. Sensitivity and Cost-Function Design

The relative weighting of cost components within the optimization objective determines how the model prioritizes packaging decisions. Common strategies include:

- **Direct cost minimization** – Emphasizes manpower and activation cost reduction.
- **Downtime minimization** – Prioritizes operational availability, sometimes at the expense of higher labor cost.
- **Multi-objective balancing** – Combines both with normalized weighting factors to achieve a cost-effective compromise.

Sensitivity analysis can identify how changes in cost parameters (e.g., labor rate, downtime penalty, activation overhead) influence packaging structure. A robust MILP formulation allows planners to simulate these trade-offs quantitatively and identify economically resilient packaging strategies — for example, when fuel prices, labor markets, or utilization rates change [6][14][31].

2.7.6. Operational Integration and Practical Benefits

When integrated into an airline's maintenance planning process, optimized packaging delivers several measurable benefits:

1. **Reduced maintenance cost** – Through minimized early-task execution and fewer redundant access actions.
2. **Higher aircraft availability** – Through reduced downtime and smoother task distribution.
3. **Improved resource planning** – Balanced packages simplify manpower and materials forecasting.
4. **Regulatory traceability** – Optimization decisions can be documented and justified through model outputs, supporting audits and AMP approval [25].
5. **Scenario evaluation** – Operators can simulate alternative packaging philosophies (e.g., equalized vs. semi-block) before implementing them operationally.

These benefits demonstrate that task packaging optimization is not only an academic exercise but also a tangible operational tool for improving efficiency and compliance in modern airline maintenance [6][14][34].

2.8. Synthesis and Conceptual Framework

The preceding sections have reviewed the evolution of maintenance task packaging from a regulatory requirement into a structured optimization problem. This synthesis consolidates those findings into a unified conceptual framework that explains *how* regulatory data, operational philosophy, and mathematical optimization interact to produce efficient, compliant, and economically viable maintenance packages.

2.8.1. Synthesis of Theoretical and Operational Foundations

Aircraft maintenance programs operate under the principle of **continuing airworthiness**, as defined by FAA and EASA regulations. The **MSG-3 methodology** and **Maintenance Review Board Report (MRBR)** establish the analytical logic that determines the set of required tasks and their maximum intervals. Manufacturers then codify this information into the **Maintenance Planning Document (MPD)**, which serves as the technical repository for all maintenance tasks.

Operators convert the MPD into their **Approved Maintenance Program (AMP)**, adapting it to their fleet utilization, environment, and reliability data. However, this conversion requires the **grouping of MPD tasks** into executable packages or checks — the *maintenance task packaging problem*.

Historically, this process relied on human judgment and heuristic grouping philosophies (block or equalized). While these methods achieved compliance, they lacked quantitative rigor, consistency, and transparency. The literature review shows that as task volume and complexity increased, manual packaging became infeasible, leading to research on algorithmic and mathematical approaches — culminating in **MILP-based optimization**.

2.8.2. Conceptual Flow: From Regulation to Optimization

The relationship between the regulatory hierarchy and the optimization framework can be expressed as a sequential flow:

Certification (MSG-3 / MRB) → MPD (Task Definitions) → AMP (Operator Packaging) → Optimization (MILP Model) → Execution (Maintenance Checks)

$\text{Certification (MSG-3 / MRB)} \rightarrow \text{MPD (Task Definitions)} \rightarrow \text{AMP (Operator Packaging)} \rightarrow \text{Optimization (MILP Model)} \rightarrow \text{Execution (Maintenance Checks)}$

3 / MRB)→MPD (Task Definitions)→AMP (Operator Packaging)→Optimization (MILP Model)→Execution (Maintenance Checks)

Each stage in this chain defines a different dimension of constraint:

- **Certification Level** – Defines *what* must be maintained and *why* (safety logic).
- **MPD Level** – Defines *when* tasks are due (intervals and thresholds).
- **Packaging Level** – Defines *how* tasks are grouped into packages (task-to-check assignment).
- **Optimization Level** – Determines *how to group tasks optimally* to minimize total cost and downtime under regulatory constraints.
- **Execution Level** – Implements packages operationally through line or base maintenance.

The MILP model developed in Chapter 3 operationalizes this flow mathematically, transforming regulatory and technical rules into solvable constraint relationships.

2.8.3. Integration of Cost and Compliance Dimensions

Optimization of maintenance task packaging is multi-dimensional: every decision involves balancing **economic** and **regulatory** imperatives. MILP models allow these dimensions to be explicitly combined:

- **Compliance constraints** (from MRB/MPD): enforce upper limits on intervals, thresholds, and applicability.
- **Operational constraints** (from AMP design): enforce access, manpower, and resource feasibility.
- **Economic objectives** (from cost modeling): minimize total labor, activation, and downtime costs.

This triad ensures that optimization outcomes remain *valid in practice*, not merely mathematically feasible. It allows operators to justify packaging decisions both economically and regulatorily — a key advantage over heuristic methods.

2.8.4. Conceptual Framework to the Research

The integrated conceptual framework for this research can be summarized as follows:

1. **Input Layer:**

- Source data from the MPD, containing interval parameters (FH/FC/calendar), access zones, man-hours, and skills.
- Regulatory and technical boundaries derived from MRBR and MSG-3 logic.

2. **Transformation Layer:**

- Data preprocessing and normalization convert heterogeneous MPD data into a structured parameter set.
- Access, interval, and cost relationships are identified and encoded.

3. **Optimization Layer:**

- MILP formulation captures task-to-package assignment decisions.
- Constraints enforce interval compliance, workload limits, and access compatibility.
- Objective function minimizes total cost (labor + downtime + activation).

4. **Output Layer:**

- Optimized task packages that are regulatorily compliant, cost-efficient, and reproducible.
- Results provide traceable justification for AMP revisions or maintenance program design.

(**Figure 2-1** (conceptual, to be added in the final thesis) could illustrate this four-layer architecture — showing data flow from MPD to optimized packaging output.)

2.9. Chapter Summary

To conclude this chapter, the essential findings are consolidated in the summary table below. Rather than repeating the detailed narrative already presented in Sections 2.2 through 2.8, the table distills the regulatory foundations, data structures, packaging philosophies, optimization framing, and research evolution into a structured overview. This format provides clarity and traceability, highlighting the progression from regulatory requirements through operational practice to optimization theory. It also emphasizes the current state of research, where heuristic and metaheuristic approaches dominate, while MILP remains comparatively under-explored but offers the most regulatorily defensible and reproducible framework. This synthesis establishes the conceptual foundation for the methodological developments presented in Chapter 3.

Level	Definition / Source	Role in Packaging	Key References
Regulation	FAA 14 CFR Part 121; EASA Part-M / Part-CAMO	Defines <i>why</i> maintenance must occur (continuing airworthiness obligations).	[1][2][25]
Methodology	MSG-3 process (EASA/IMRBPB)	Reliability-centered logic that determines <i>what</i> preventive tasks and intervals are required.	[3]
Certification	Maintenance Review Board Report (MRBR)	Codifies MSG-3 outputs into certified maintenance requirements for each aircraft type.	[4][25]
Technical Data	Maintenance Planning Document (MPD)	Provides the full technical listing of tasks, intervals, access, skills, and metadata — the raw data for packaging.	[5][23][28]
Operator Program	Approved Maintenance Program (AMP)	Operator-specific implementation of MPD tasks, adapted to utilization, environment, and reliability.	[11][23]
Packaging Philosophies	Block and Equalized Programs	Historical heuristic grouping methods; feasible but inefficient, inconsistent, and non-optimal.	[11][23][20][21][22]
Optimization Framing	Packaging as a constrained assignment problem	Defines packaging as discrete task-to-event decisions under regulatory and cost constraints.	[6][8][14][24]

Level	Definition / Source	Role in Packaging	Key References
Optimization Model	Mixed-Integer Linear Programming (MILP)	Provides a deterministic, auditable, and regulator-compatible framework for packaging optimization.	[6][8][14][17][18][24][25]
Economic Dimension	Direct, indirect, and opportunity costs (labor, activation, downtime)	Embeds cost trade-offs (early execution, activation frequency, workload balance) into the optimization objective.	[11][12][13]
Research Evolution	Majority of published studies emphasize heuristics, metaheuristics, and hybrid GA-based methods (e.g., GA, SA, GA-MILP hybrids) to demonstrate feasibility [6][8][9][10][19][31][32][34]. Pure MILP applications remain limited but are emerging as the most regulatorily defensible and auditable approach [6][8][14][24].	Shows the field's trajectory: manual heuristics → metaheuristics/hybrids (dominant) → limited but growing MILP research.	[6][7][8][9][10][14][19][31][32][34]
Conceptual Framework	Integrated regulatory–data–optimization chain	Links regulation, MPD data, optimization theory, and practical	[6][8][14][24]

Level	Definition / Source	Role in Packaging	Key References
		implementation; foundation for Chapter 3.	

Chapter 3

Methodology

3.1. Overview of Research Methodology

This section presents the overall research methodology framework that guides the development, formulation, and implementation of the optimization model for aircraft maintenance task packaging. The framework integrates quantitative modeling, computational implementation, and digital system design into a unified methodological structure aligned with the research objectives described in Chapter 1.

The research adopts a **quantitative optimization approach**, specifically a **Mixed-Integer Linear Programming (MILP)** framework, to identify the most cost-effective combination of maintenance tasks to group into packages. The methodology was designed to ensure that the resulting optimization model is both *academically rigorous* and *operationally applicable* within real-world Maintenance, Repair, and Overhaul (MRO) environments.

3.1.1. Methodological Approach

The study follows a **deductive research approach**, beginning from theoretical principles of operations research and optimization, then applying these principles to the empirical context of aircraft maintenance data. The overall methodological sequence is structured as follows:

1. **Theoretical Modeling Layer** – Construction of the MILP mathematical model, defining variables, objective functions, and constraints based on maintenance logic and regulatory compliance requirements (FAA, 2016; EASA, 2020).
2. **Computational Layer** – Translation of the mathematical model into executable form using Python and Gurobi, including data preprocessing, model generation, and solver execution.
3. **System Integration Layer** – Development of a modular architecture that integrates data handling, optimization computation, and output visualization, designed for compatibility with MRO systems (AMOS, RAMCO).

This multi-layered approach allows the research to move systematically from **conceptual modeling** → **mathematical formulation** → **practical implementation**, ensuring full methodological traceability from theory to result.

3.1.2. Research Flow and Integration Structure

The integrated methodological flow is depicted in **Figure 3-1**, illustrating the interaction between research phases, data, model, and system components.

Each phase contributes to one or more subsections in Chapter 3 and links to supporting appendices for mathematical and architectural detail.

(Figure 3-1 illustrates the overall research methodology framework, showing how theoretical modeling, computational implementation, and system integration interact within the research process. Appendices A and B serve as detailed extensions of the mathematical and architectural layers, respectively)

3.1.3. Methodological Scope and Boundaries

The scope of this methodology covers the development of an **optimization model for maintenance task packaging** within the planning horizon of a single aircraft. The research focuses on modeling **how tasks are grouped into packages**, not the scheduling of these packages over time. The scope is therefore defined by the following boundaries:

Boundary Type	Scope Definition	Exclusion
Analytical Scope	Optimization of maintenance task packaging using MILP	Scheduling of maintenance packages over time
Operational Scope	Single-aircraft model derived from MPD data	Multi-aircraft or fleet-level optimization
Computational Scope	Deterministic parameters, discrete time horizon	Stochastic or continuous-time formulations
System Scope	Integration with MRO/AMP digital systems	Real-time maintenance execution control

These boundaries are consistent with the methodological assumptions defined in Section 3.3.6 and ensure that the research remains focused on the **task packaging problem**, which is distinct from scheduling or resource allocation problems (Marvella, 2023; Witteman, 2021; Filho et al., 2024).

3.2. Data Framework and Model Inputs

The data framework defines the structure, source, and preprocessing logic of all information used in constructing the MILP model. Because aircraft maintenance data are complex, hierarchical, and highly regulated, this framework ensures that each data element—

ranging from MPD tasks to cost and capacity parameters—is properly normalized and mapped to the corresponding model and system variables.

The data framework serves two essential purposes:

1. To provide **quantitative inputs** for the MILP model (see Section 3.3 and Appendix A);
2. To establish **structured data flows** for computational processing and visualization (see Section 3.7 and Appendix B).

3.2.1. Data Sources and Structure

The primary dataset used in this research originates from the **Airbus A320 Family Maintenance Planning Document (MPD)** and related **Maintenance Review Board Report (MRBR)** (Airbus, 2022; FAA, 2016; EASA, 2020). The dataset consists of individual maintenance tasks defined by the manufacturer and approved by regulatory authorities. Each task includes critical attributes such as:

- Task ID and description
- Interval (in Flight Hours, Flight Cycles, and/or Calendar time)
- Threshold for first performance
- Access zone or panel
- Estimated man-hours
- Skill code (e.g., airframe, avionics, powerplant)
- Applicability (effectivity)

These MPD fields are consistent with the structural hierarchy summarized in **Table 2-1** in Chapter 2, which shows how each field contributes to the optimization parameters. The dataset represents one aircraft unit, consistent with the single-aircraft scope of this study.

For confidentiality, only non-proprietary task data are used; numerical examples presented later are representative and anonymized. The selected scope ensures that the model formulation remains scalable and generalizable to various fleet types.

3.2.2. MPD Data Characteristics and Preprocessing

The MPD dataset is hierarchical by nature, grouping tasks according to ATA chapters, task numbers, and functional zones. For optimization purposes, these data are **flattened** into a normalized table where each task becomes a unique record with standardized parameters.

A representative data structure is shown below:

Task ID	ATA Code	Interval (FH)	Threshold (FH)	Zone	MH	Access Type	Skill Code
27-31-11-000-001	27	600	1200	531	3.5	Wing/Flap	A&P
32-22-00-100-002	32	800	1600	572	2.5	Nose/Bay	Avionics
29-12-05-200-004	29	400	1200	121	1.5	Fuselage	A&P

Preprocessing steps:

1. **Data cleaning:** Remove incomplete or obsolete tasks.
2. **Normalization:** Convert all intervals and thresholds to a single consistent unit (Flight Hours).
3. **Access mapping:** Assign access zone labels (z_i) and classify according to shared access categories.
4. **Parameter derivation:** Compute man-hour equivalents (MH_i) and assign regulatory limits (h_i).
5. **Filtering:** Exclude non-routine or condition-based tasks outside the MPD scope.

These steps prepare the dataset for conversion into model indices and parameters (Section 3.2.3).

3.2.3. Model Parameter Generation

After preprocessing, the structured MPD data are transformed into the key parameters required for MILP modeling. This transformation converts physical and economic attributes into **quantitative decision elements** used in the optimization process.

Symbol	Definition	Source / Basis	Unit
$i \in I$	Task Index	MPD data (Identifier for each maintenance task)	Integer
(h_i)	Interval limit for task i	MPD interval	FH
(MH_i)	Man-hour requirement for task i	MPD	MH

Symbol	Definition	Source / Basis	Unit
 (z_i)	Access zone identifier	MPD zone number	Categorical
(ϕ)	Dabor cost coefficient	Derived from (labor and material)	Dimesionless
(r_{lab})	Labor rate cost	Based on industry benchmarks (Naveo, 2024; FL Technics, 2022)	USD
(ρ)	Revenue rate	Based on line downtime rate (Aircraft Commerce, 2023)	USD/hour
 (ω)	Activation cost per package	Based on line downtime rate (Aircraft Commerce, 2023), Derived from fixed activation rate	USD/event

These parameters form the quantitative basis of the optimization model presented in **Section 3.3**, where they define the coefficients and constraint limits of the MILP equations.

3.2.4. Variable Clasification

To improve methodological transparency, all model variables are classified based on their research and functional roles—distinguishing between **decision**, **control**, **dependent**, and **outcome** variables. This classification facilitates academic traceability and aligns the optimization model with standard quantitative research design.

Variable Type	Symbol / Example	Definition / Function	Role in Model
Decision Variables	$s_a^A, s_c^C, x_{i,t}, v_a^A, v_c^C$	Represent the optimizer’s controllable choices, such as the selection of A- and C-check base intervals and the assignment of tasks to bins or line maintenance.	Determined by the MILP solver to minimize total cost (model output structure and feasibility)
Control Variables (Model Parameters)	$r_{lab}, r_{down}, \phi, \rho, \omega, h_i, MH_i, h_i^{max}$	Define system settings (labor rate, interval limits, activation overhead, penalty	Ensure consistency with planning regulations and

Variable Type	Symbol / Example	Definition / Function	Role in Model
		coefficients, man-hour maps, allowable thresholds)	realistic airline data.
Dependent Variables (Derived Outputs)	$z_{a,t}^A, z_{c,t}^C, e_{i,k}, t_{i,k}$	Derived automatically from decisions (e.g., workload per bin, early slack values, or check activation flags).	Reflect operational consequences of chosen intervals.
Outcome Variables (Performance Indicators)	Z , matrix of task – bin assignment, L_i, A, C	Represent measurable results such as total cost, workload distribution, and evaluation metrics for the number of packages (interval compliance).	Used for model validation and performance assessment

The binary decision nature of $s_a^A, s_c^C, x_{i,j}, v_a^A, v_c^C$ forms the core of the MILP formulation (see Section 3.3 and Appendix A). Control variables serve as fixed constraints or coefficients, while dependent and outcome variables provide post-optimization interpretive metrics for cost and performance analysis.

This classification framework ensures that the optimization process is not treated as a “black box” but as a logically decomposed system whose internal behavior is fully explainable and regulatorily traceable (Bertsimas & Tsitsiklis, 1997; Vielma, 2015)

(Figure 3.2 Backup for Figure List)

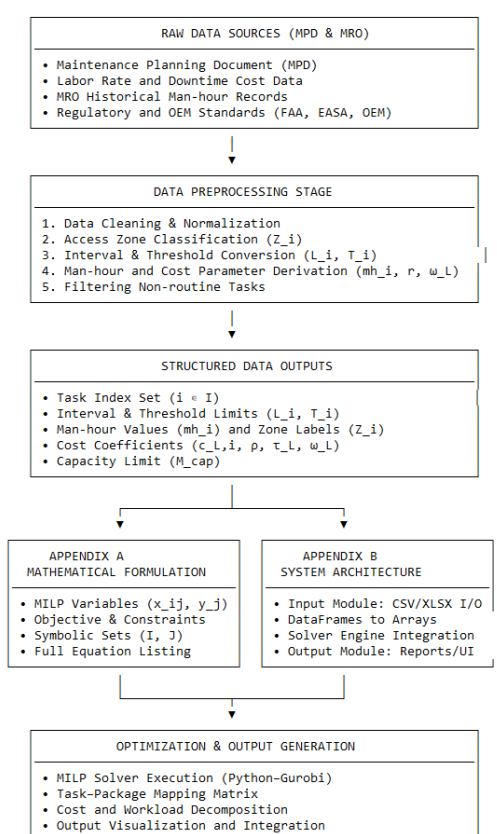


Figure 3-2 illustrates the data-to-model mapping process, showing how raw maintenance and cost data from MPD and MRO sources are preprocessed, normalized, and transformed into structured parameters for the MILP optimization model (Appendix A) and computational system (Appendix B). The structured data ensure consistency between the theoretical model and its digital implementation.

3.3. Model Formulation

This section presents the development of the **Mixed-Integer Linear Programming (MILP)** model used to optimize maintenance task packaging. The model aims to determine the optimal grouping of maintenance tasks into packages that minimize total maintenance cost while adhering to regulatory, operational, and capacity constraints.

It represents a continuation and enhancement of the optimization framework developed by Marvella (2023) for the Boeing 737-800 NG, with extensions in variable design, cost decomposition, and computational generality for multi-package evaluation.

3.3.1. Objective Model

The principal objective of the model is to **minimize total maintenance cost**, expressed as a function of:

1. **Labor cost** (man-hours multiplied by rate),
2. **Activation cost** (fixed cost per maintenance package), and
3. **Downtime cost** (cost of aircraft unavailability).

This optimization problem can be stated conceptually as:

$$Z = C_{Direct} + C_{Downtime} + C_{Overhead} + C_{Early} + C_{Line} + C_{Balance} \quad (2)$$

subject to task grouping, capacity, and interval compliance constraints.

Each term correspond to a distinct operational phenomenon :

1. Direct Maintenance Cost

$$C_{Labor} = \sum_{i \in I} \sum_{t \in T} \phi \cdot r \cdot mh_i \cdot x_{i,t} \quad (3)$$

This term reflects the uplifted labor cost of assigning tasks to bins, driven by the independent assignment variables.

2. Downtime Cost

$$C_{Downtime} = \sum_{t \in T} \rho \cdot (d_A \cdot v_t^A + d_C \cdot v_t^C) \quad (4)$$

This captures the opportunity cost of aircraft unavailability, determine by visit activation decisions.

3. Overhead Cost

$$C_{Overhead} = \sum_{t \in T} (\omega_C \cdot v_t^C + \omega_A \cdot u_t^{A-only}) \quad (5)$$

Each maintenance visit incurs a fixed overhead, with C-checks dominating when both occur simultaneously.

4. Early Execution Penalty

$$C_{Early} = \rho \cdot \sum_{i \in I} \sum_k e_{i,k} \quad (6)$$

This penalizes premature execution of tasks, ensuring that bin consolidation does not come at the expense of excessive early pulls.

5. Line Maintenance Cost

$$C_{Line} = \sum_{i \in I} L_i \cdot occ_i \cdot (\phi \cdot r \cdot mh_i + \rho \cdot \tau_L + \omega_L) \quad (7)$$

Tasks that cannot be accommodated within the chosen A interval are relegated to line maintenance, incurring additional per-occurrence costs.

6. Workload Balance Penalty

$$C_{Balance} = \lambda \cdot W_{peak} \quad (8)$$

This optional term penalizes excessive concentration of manhours in a single bin, encouraging smoother workload distribution.

The optimization logic seeks the **minimum-cost packaging configuration** that satisfies all maintenance interval and regulatory requirements. Unlike maintenance scheduling models, this approach focuses exclusively on **task grouping**, where the solver determines **which tasks belong together**, not when they will be executed.

The mathematical structure of the model is linear and deterministic. Decision variables are binary, representing whether a task i is assigned to a active visit (package) t . Continuous variables are used only for cost and resource accumulation, ensuring linearity throughout the model.

The general structure is as follows (details in Appendix A):

$$\text{Minimize } Z = \underbrace{\sum_{i \in I} \sum_{t \in T} c_i \cdot x_{i,t}}_{\text{Tasks Individually}} + \underbrace{\sum_{t \in T} (\rho \cdot d + \omega) \cdot v_t}_{\text{Package/ Visit}} + \underbrace{\left(\rho \cdot \sum_{i \in I} \sum_k e_{i,k} \right)}_{\text{Penalties}} \quad (9)$$

where can be expressed by :

$$c_i = \phi \cdot r \cdot MH_{I_i} \quad (10)$$

This cost structure provides a comprehensive economic view of maintenance packaging by capturing both direct (labor, downtime) and indirect (overhead, early penalty) components. The formal algebraic expressions of each term are presented in **Appendix A.2** (Eqs. A1), where parameter definitions (labor rate, downtime coefficient, penalty weights, etc.) are also given.

Cost Component	Description	Model Interaction
C_{Direct}	Direct labor cost proportional to man-hours assigned in each bin[7],[8].	Linked to task–bin assignment $x_{i,t}$.

$C_{Downtime}$	Cost incurred from aircraft being unavailable during scheduled checks[8],[9].	Aggregated over active A/C checks per bin.
$C_{Overhead}$	Logistical and administrative overhead applied per check event.	Computed from activation variables v_t^A, v_t^C .
C_{Early}	Penalty for early execution of tasks relative to threshold interval[4],[10].	Derived from early-slack variable $e_{i,k}$.
C_{Line}	Cost of transferring infeasible tasks to line maintenance[4],[10].	Controlled by binary flag L_i .
$C_{Balance}$	Penalty for unbalanced workload across bins.	Determined by the peak workload variable W_{peak} .

3.3.2. Model Assumption (Limitation explain in detail)

The model is formulated under a defined set of academic and practical assumptions to ensure solvability, clarity, and industrial realism. These assumptions are summarized below.

Assumption Type	Description	Rationale
Discrete Time Horizon	Tasks occur within discrete package cycles. Only flight-hour accumulation is considered as the controlling variable for interval limits. Calendar-based deterioration can be incorporated in future hybrid models	Maintenance events are pre-defined by planning intervals.
Single-Aircraft Scope	Model applies to a single representative aircraft. Fleet-level interactions such as resource sharing or hangar allocation are outside the current scope.	Focused analysis and data validation; avoids fleet-level complexity.
Deterministic Parameters	All input parameters (costs, man-hours, intervals) are deterministic. Task durations, man-hours, and labor rates are assumed deterministic and known in advance. Uncertainty in execution times or stochastic	Aligns with standard MILP modeling and available data.

	resource availability is excluded to retain linear structure	
Static Capacity	Maintenance-facility capacity W_{max} is fixed across all bins; variable staffing or shift adjustments are not modeled	Simplifies workload balancing under fixed man-hours assumptions.
Independent Tasks	Each maintenance task is treated as functionally independent; precedence or technical coupling effects are omitted.	Enables pure task packaging optimization. Consistent with MPD task-based planning, where tasks are modular by design.

These assumptions are consistent with both theoretical MILP solvability and MRO operational practice. A continuous or stochastic model would introduce additional uncertainty parameters and require scenario-based solvers, which are beyond the intended deterministic scope of this research (Filho et al., 2024).

3.3.3. Simplification and Justification for Discrete MILP Design

The **discrete formulation** of this model is a deliberate methodological choice. While continuous or stochastic models can capture dynamic uncertainty, the **maintenance packaging problem** is inherently discrete because:

1. **Maintenance packages are discrete events** — A-checks, C-checks, and bridging checks occur at fixed cycles, not continuously.
2. **Tasks are indivisible entities** — Each task must be fully assigned to one package, not partially distributed.
3. **Regulatory compliance requires discrete thresholds** — Intervals and thresholds (e.g., 400 FH, 1200 FH) are defined in quantized units.
4. **Solver interpretability and integration** — Discrete MILP output maps directly into MRO system databases and maintenance planning tools (AMOS, RAMCO).

Despite the simplifications, the chosen modeling strategy provides several methodological and practical advantages.

Advantage	Description
-----------	-------------

Computational Efficiency	The discrete MILP can be solved to global optimality using commercial solvers within practical time limits.
Interpretability	Each decision variable (e.g., selected A-interval) has a direct operational meaning and can be validated by domain experts.
Regulatory Compliance	Discrete interval bins align with actual MPD practices and authority-approved check cycles.
Scalability	The model structure supports incremental dataset expansion without altering the formulation.
Transparency and Replicability	All variables, parameters, and constraints are explicitly defined (Appendix A), enabling full peer verification.

In summary, these assumptions and simplifications establish a robust methodological foundation for the optimization framework. They enable the model to balance realism and solvability—an essential trade-off in large-scale maintenance scheduling research—and ensure that results are both computationally rigorous and operationally actionable

3.3.4. Mathematical Model Summary

The full MILP mathematical formulation—including complete variable sets, objective functions, constraint equations, and notation tables—is presented in **Appendix A**. Only the summarized structure is shown here for contextual clarity (As expressed in Equation (9) and (10) in Section 3.3.1):

$$\text{Minimize } Z = \underbrace{\sum_{i \in I} \sum_{t \in T} c_i \cdot x_{i,t}}_{\text{Tasks Individually}} + \underbrace{\sum_{t \in T} (\rho \cdot d + \omega) \cdot v_t}_{\text{Package/ Visit}} + \underbrace{\left(\rho \cdot \sum_{i \in I} \sum_k e_{i,k} \right)}_{\text{Penalties}}$$

where can be expressed by :

$$c_i = \phi \cdot r \cdot MH_{I_i}$$

Subject to :

$$\sum_{a \in A} s_a^A = 1, \quad \sum_{c \in C} s_c^C = 1, \quad (\text{interval selection}) \quad (11)$$

$$\sum_{t \in T} x_{i,t} + L_i = 1, \quad (\text{task assignment}) \quad (12)$$

$$\begin{aligned} v_t^A &\leq z_t^A, & v_t^C &\leq z_t^C, \\ v_t &\leq v_t^A + v_t^C, \end{aligned} \quad (\text{package activation logic}) \quad (13)$$

$$x_{i,t} \leq v_t^A + v_t^C.$$

$$\sum_{t \leq s} x_{i,k+1,t} \leq \sum_{t \leq s} x_{i,k,t}, \quad (\text{execution order}) \quad (14)$$

$$s_a^A, s_c^C \in \{0, 1\}, v_t^A, v_t^C \in \{0, 1\}, \quad (\text{binary decision}) \quad (15)$$

Each constraint directly corresponds to a real-world maintenance rule or operational boundary.

Constraint Family	Conceptual Purpose	Real-world Interpretation
Interval Selection	Ensures only one A- and one C-check base interval are chosen.	Represents operator policy or regulatory selection of nominal intervals.
Availability Logic	Activates check bins according to the selected base intervals[4].	Generates calendar structure (A1, A2, C1, etc.).
Visit Hierarchy	Enforces dominance of C-check over A-check when they coincide.	Reflects that C includes A by procedural definition.
Task Assignment	Allocates each task to exactly one bin or to line maintenance.	Ensures exclusivity of task occurrence.
Rolling Max-Gap Constraint	Maintains compliance with maximum threshold intervals[10].	Guarantees interval reset and regulatory adherence.
Early-Execution Slack	Measures and penalizes premature execution before threshold.	Models operational inefficiency or opportunity loss.
Workload and Capacity	Balances man-hour distribution per bin within maximum capacity.	Prevents excessive load on maintenance facilities.

The extended equation set—including additional access zone, interval, and early-execution constraints—is fully enumerated in **Appendix A**, along with a comprehensive symbol table and dimensional definitions.

3.4. Model Implementation

This section describes how the formulated MILP model for maintenance task packaging (Section 3.3) is implemented computationally. The implementation transforms the mathematical model into an executable Python–Gurobi program, ensuring the model can process MPD-based data, solve the optimization problem efficiently, and generate interpretable outputs for engineering analysis and integration with MRO systems.

The design follows a **modular implementation architecture**, emphasizing flexibility, transparency, and reproducibility—essential qualities for optimization-based decision support systems (Li et al., 2022; Filho et al., 2024).

3.4.1. Implementation Environment

The computational environment consists of three key components:

Component	Specification	Purpose
Programming Language	Python (version ≥ 3.10)	General model scripting and data processing
Optimization Solver	Gurobi Optimizer (version ≥ 10.0)	Solves MILP using branch-and-bound and presolve algorithms
Libraries and Frameworks	NumPy, pandas, PuLP/GurobiPy, Matplotlib	Data handling, optimization modeling, and visualization

The implementation adheres to open standards for reproducibility. Python’s GurobiPy interface is used to build and solve the MILP problem directly, allowing symbolic variable declaration, linear constraint generation, and cost function construction within a unified environment.

Solver settings are tuned for **medium-scale optimization problems**, typically involving 100–500 MPD tasks and 8–12 candidate maintenance packages, ensuring convergence within reasonable computational time (< 5 minutes on standard workstation hardware).

3.4.2. Model Translation and Workflow Overview

The implementation translates the conceptual model into a structured workflow consisting of **four main modules**, which together form the **computational pipeline**.

This structure parallels the layered system architecture presented in **Appendix B**, ensuring full consistency between the theoretical model (Appendix A) and its digital realization.

Module 1 – Input Handling

- Reads MPD, cost, and configuration data from CSV/XLSX sources.
- Validates field completeness (intervals, man-hours, access zones).
- Converts raw tabular data into Python dataframes and indexed arrays.
- Ensures consistent data dimensions for MILP variable creation.

Module 2 – Model Builder

- Initializes the Gurobi environment.
- Defines all **decision variables** ($x_{i,t}$) and **parameters** (cost, capacity).
- Constructs the **objective function** and **constraint sets** dynamically based on data size.
- Encapsulates constraint logic for modularity (e.g., task coverage, capacity, activation).

Module 3 – Solver Execution

- Configures solver parameters:
 - MIPGap = 0.001 (for near-optimality tolerance)
 - TimeLimit = 600 seconds (to prevent excessive runtime)
 - Presolve = 2 (for model simplification before branching)
- Executes the optimization using the **branch-and-bound** method.
- Extracts solver logs for performance tracking (iterations, node count, runtime).
- Captures optimal values of $x_{i,t}$, and total cost Z .

Module 4 – Output Generation

- Converts solver outputs into readable **Task–Package Mapping Matrices**.
- Computes total and component-wise costs (labor, activation, downtime).
- Exports results to structured Excel/CSV and visual dashboards.
- Feeds outputs into the **Post-Processing Layer** (Appendix B, Module 4) for visualization.

3.4.3. Solver Configuration and Optimization Process

The **Gurobi Optimizer** is configured to ensure convergence, stability, and repeatability. Key computational configurations are summarized below:

Parameter	Setting	Purpose
MIPGap	0.001	Ensures high-quality solution ($\leq 0.1\%$ from global optimum)
Presolve Level	2	Eliminates redundant constraints and tightens model bounds
Branching Strategy	Automatic	Adapts between depth-first and best-bound search
Heuristics Level	0.1	Speeds up convergence through feasible solutions
Time Limit	600 s	Ensures manageable computation time
Threads	4–8 (auto)	Utilizes multi-core processing efficiently

This setup is designed to balance computational cost and result precision, enabling practical runtime while maintaining academic rigor in solution quality.

3.4.4. Implementation Flow Diagram\

The process flow of model implementation is illustrated in **Figure 3-4**, showing the sequential interaction of the four modules within the overall optimization system.

Figure 3-4 illustrates the model implementation flow, showing how the MILP formulation is executed within a modular Python–Gurobi environment. The sequence of modules transforms MPD and cost data into optimization results, ensuring transparency and traceability from input to output.

3.5. Model Output Design and Interpretation

The output design defines how the results of the optimization process are structured, represented, and interpreted. Outputs generated by the MILP model provide actionable insights for decision-makers, enabling the identification of optimal maintenance task groupings while maintaining transparency in cost and workload evaluation.

The design of the model outputs follows three primary objectives:

1. **Interpretability** – Results must be clear and directly usable by maintenance planners.
2. **Traceability** – Each output value must correspond to verifiable input data and model logic.

3. **Integrability** – Outputs must be compatible with digital MRO systems for potential implementation.

3.5.1. Overview of Output Categories

The MILP model produces four main categories of outputs: **decision**, **analytical**, **performance**, and **visual** results. These outputs correspond to distinct layers of model interpretation and are designed for both academic validation and industrial usability.

Output Category	Description	Derived From	Purpose
Selected Base Intervals (A^*, C^*)	Optimal A- and C-check base intervals selected by the model.	Binary variables s_a^A, s_c^C	Defines the periodic structure of the maintenance program.
Task–Bin Assignment Matrix	Binary matrix showing which task i is performed in bin t .	$x_{i,t}$	Enables direct visualization of task distribution across checks.
Line-Maintenance Set	Subset of tasks allocated to line maintenance.	Binary variable L_i	Represents tasks exceeding interval limits or not aligned with base checks.
Cost Breakdown	Decomposition of total cost Z into its components.	Eq. A2–A7	Quantifies trade-offs among labor, downtime, overhead, early penalty, and line cost.
Workload Distribution	Man-hour aggregation per bin, including peak workload W_{peak} .	$W_{peak}, x_{i,t}, MH_i$	Evaluates resource balance and utilization efficiency.
Solution Validation Indicators	Metrics verifying feasibility, optimality, and robustness.	Eq. A21–A23	Confirms that numerical solutions meet methodological acceptance rules.

Each output type corresponds directly to a modeling layer discussed in 3.2 and 3.3. Together, these outputs create a multidimensional analytical view that connects model decisions with practical maintenance implications.

3.5.2. Task-Package Mapping Matrix

The **Task–Package Mapping Matrix** ($X_{i,t}$) is the primary decision output of the optimization. It defines which maintenance tasks are grouped together within each package, forming the core analytical result of the MILP model.

A simplified conceptual example is shown below:

Task ID (i)	Pkg A1	Pkg A2	Pkg A3	Pkg C1	Selected Package
27-31-11-001	1	0	0	0	A1
32-22-00-002	0	1	0	0	A2
29-12-05-004	0	0	1	0	A3
53-41-07-006	0	0	0	1	C1

Each “1” indicates a discrete assignment of a task i to a package t , forming the binary structure of the MILP decision variables. This matrix becomes the basis for computing cost, capacity, and operational efficiency in subsequent analyses.

3.5.3. Cost and Workload Decomposition Outputs

The optimization model decomposes total maintenance cost as expressed by Equation (9) and explained in Section 3.4. That formula can be categorized into **three major components** as expressed by Equation (9) and can be defined in general by :

$$Z = C_{Task} + C_{Package} + C_{Penalties} \quad (16)$$

where:

- C_{Task} = Direct cost component (sum of labor cost and material cost),
- $C_{Package}$ = Package activation cost component (downtime and overhead),
- $C_{Penalties}$ = Penalty formulation cost component.

A typical decomposition summary is presented by dummy outputs below (as expected and illustrated output) :

Package	Task Individual Cost (\$)	Package Cost (\$)	Penalties Cost (\$)	Total (\$)	Workload (MH)
A1	5,200	1,000	800	7,000	210
A2	4,950	1,000	800	6,750	200
A3	5,100	1,000	800	6,900	215
C1	12,800	2,500	1,600	16,900	410
Total	27,950	5,500	4,000	37,450	1,035

The cost decompositions presented here constitute a subset of the overall objective cost model, serving as an illustrative example to demonstrate the system’s design and to provide insight into cost drivers and workload distribution across packages. They also serve as validation metrics for comparing different optimization configurations (e.g., heuristic baseline vs. MILP-based packaging), as discussed in Section 3.6.

3.5.4. Discrete Output Design Rationale

The outputs of this research are **discrete**, not continuous, by deliberate methodological choice. This design aligns with the **nature of maintenance events**, **data availability**, and **integration feasibility**, as mention in Section 3.3.3 and re-explained below.

Justification	Explanation
Operational Reality	Maintenance packages are inherently discrete — A1, A2, C1, etc. There is no meaningful partial allocation of a task between packages.
Regulatory Compliance	Maintenance intervals and thresholds are expressed in fixed discrete units (e.g., 400 FH, 800 FH). Continuous values would have no regulatory meaning.
Data Resolution	MPD and MRO data are defined in discrete time increments, making continuous outputs infeasible.
Solver Interpretability	Discrete (binary) outputs are directly interpretable and can be validated against real-world check families.
System Integration	MRO databases use discrete identifiers for maintenance packages and task allocations; continuous variables would be incompatible with these systems.

Thus, discrete output design ensures **academic consistency**, **industrial applicability**, and **system compatibility**—three dimensions essential for bridging research and practice.

3.5.5. Line Maintenance Task Classification

The optimization model focuses on the packaging of *A-check* and *C-check* tasks, which are periodic and require hangar-level planning. However, certain low-interval tasks categorized under *Line Maintenance* (e.g., daily, transit, or overnight checks) are not subject to optimization but are included in the system for classification purposes.

These tasks are automatically filtered during preprocessing (Module 2) and stored separately during post-processing (Module 4) to enable complete reporting and cost breakdowns. This ensures that the final analytical outputs reflect the full maintenance program composition, even though optimization is applied only to periodic packages.

3.5.6. Output Visualization and Interpretation

Visualization modules are implemented in the **post-processing layer** (Appendix B, Module 4) to transform solver results into interpretable visual formats. Typical visualizations include:

- 1. **Cost Decomposition Charts** – Bar charts illustrating labor, activation, and downtime proportions per package.
- 2. **Workload Distribution Graphs** – Visual comparison of man-hour loads among A- and C-check packages.
- 3. **Task–Package Allocation Maps** – Color-coded matrices showing task assignments across packages.
- 4. **Sensitivity Analysis Plots** – Illustrating the impact of capacity or cost coefficient variation on total cost.

Beyond raw outputs, the interpretation phase connects optimized results with research objectives and theoretical contributions:

Analytical Question	Model Element	Interpretation Logic
How are optimal intervals determined?	s_a^A, s_c^C	The solver selects intervals minimizing total cost while respecting threshold logic.
How efficiently are tasks grouped?	$x_{i,t}$ matrix	Dense task clustering indicates effective bundling and minimal early penalties.

What drives total cost reduction?	Cost decomposition (Eq. A1)	Identify which component exhibits the largest change compared to baseline.
Is workload evenly distributed?	W_{peak} and workload curve	Balanced workloads indicate effective equalization.
Are results stable and reproducible?	Robustness test (Eq. A23)	Consistency under parameter perturbations confirms robustness.

This analytical mapping allows conclusions in the **Results and Analysis** chapter to be systematically tied back to methodological mechanisms. The generation of task–package matrices, workload histograms, and cost-composition charts is automated through post-processing scripts contained in the output layer of the system architecture (Appendix B.4).

3.6. Model Validation and Evaluation

Validation ensures that the develop optimization model and its outputs are both mathematically sound and operationally credible. In line with the research objectives, this section defines the procedures used to confirm model feasibility, asses optimality, test robustness, and evaluate the practical validity of the resulting maintenance packaging design.

The evaluation framework adopts a **dual-layer validation approaches**:

1. **Quantitative Validation** — verifies that numerical results meet solver-based, logical, and statical consistency conditions.
2. **Qualitative Validation** — assesses interpretability, practicality, and conformity of generate task packages with real maintenance-engineering judgement.

This structure aligns with established practice in applied operation research and decision-support system development (Reference).

3.6.1. Quantitative Validation

Quantitative validation focuses on three principal criteria: feasibility, optimality, and robustness. These criteria are evaluated using solver diagnostics, constraint satisfaction checks, and sensitivity testing.

a) Feasibility Verification

A feasible solution must satisfy all model constraints within a numerical tolerance of 10^{-6} .

$$|A_c x - b_c| \leq 10^{-6}, \forall c \in C$$

(A22 revisited)

As defined in Appendix A.5.

This condition guarantees that all interval, assignment, and capacity rules are respected. During post-processing, every constraint violation is automatically logged; any infeasibility triggers re-optimization with adjusted parameters. This verification follows recommended practices in mathematical programming to guarantee solution integrity [3].

b) **Optimality Criterion**

Solver-reported optimality is confirmed using the Mixed-Integer Programming gap (MIPGap):

$$MIPGap = \frac{|Z_{best} - Z_{bound}|}{|Z_{best}|} \times 100\% \leq 1\%$$

A solution is **accepted** when the termination status is *Optimal* or *Feasible*, and the MIPGap does not exceed 1%. Solver configuration parameters are documented in Eq. A21 of Appendix A.

This ensures that the accepted solution lies within 1% of the global optimum while maintaining computational practicality. This criterion aligns with standard MILP solution quality definitions in optimization literature [1, 2] and with industrial practice in scheduling and resource-allocation models.

c) **Robustness Assessment**

To test stability, key parameters—labor rate, downtime rate, and early penalty coefficient—are perturbed by $\pm 10\%$.

Let Z^+ and Z^- denote objective values under upper and lower perturbations.

The relative variation is computed as :

$$\Delta Z_r = \frac{|Z^+ - Z^-|}{Z^*} \leq 0.05, \quad (\text{A23 revisited})$$

Where Z^* is the baseline optimum.

If the same base interval (A^*, C^*) and check structure persist, the solution is deemed robust. This sensitivity analysis validates that minor cost or interval fluctuations do not alter the optimal configuration—a requirement for operational deployment.

d) **Computational Efficiency Metrics**

Performance indicators such as total solver runtime, number of constraints, and variable counts are also recorded to evaluate scalability. These metrics support the discussion of

model tractability and provide empirical evidence of computational feasibility for real MPD datasets.

The numerical validation routines—feasibility scanning, optimality-gap tracking, and robustness perturbation tests—are implemented programmatically within the same Python environment, as detailed in **Appendix B.3**.

3.6.2. Qualitative Validation

Quantitative validity alone does not guarantee operational applicability. Therefore, a complementary **expert-based qualitative validation** is conducted to assess interpretability and conformity with maintenance-engineering logic.

a) Expert Review of Task-Check Assignments

Maintenance engineers or planning specialists review the **task–bin assignment matrix** generated accordance to 3.5. They verify whether:

- Task groupings comply with standard A/C-check logic,
- Regulatory inspection intervals are respected, and
- No impractical clustering (e.g., incompatible access requirements) occurs.

Feedback from this review provides **face validity**, ensuring that the optimization output aligns with domain intuition and real-world constraints **(Reference)**

b) Comparison with Existing Practices

The optimized maintenance packaging is compared with the baseline **block** or **equalized** packaging method[4],[5]. Performance differences are evaluated across:

- Total cost reduction,
- Workload balance improvement, and
- Early-execution penalty reduction.

Such comparative analysis provides **construct validity** by demonstrating that the proposed model not only reproduces but improves upon existing planning logic.

c) Result Interpretation Session

In practical validation workshops, stakeholders discuss the interpretability of the optimization outputs—particularly the cost decomposition and workload curves. Observations are documented to refine parameters or thresholds for future iterations.

3.6.3. Validation Summary and Acceptance Rules

The combination of quantitative and qualitative procedures produces a comprehensive evaluation matrix summarized below.

Validation Dimension	Criterion	Measurement / Reference	Acceptance Threshold
Feasibility	Constraint satisfaction	Eq. (A22)	Max violation $\leq 10^{-6}$
Optimality	Solver gap	Eq. (A21)	MIPGap $\leq 1\%$
Robustness	Parameter sensitivity	Eq. (A23)	$\Delta Z \leq 5\%$, structure stable
Expert Review	Task-check logic	Domain evaluation	Conforms to MPD standards
Comparative Improvement	Cost and workload	Baseline vs. optimized [4],[5]	$\geq 5\%$ cost reduction or improved balance
Interpretability	Visualization clarity	Assignment matrix, workload plot	Readable and traceable to variables

A solution is formally **accepted** when it satisfies all quantitative criteria and passes at least two qualitative review stages (expert validation + baseline comparison). Accepted solutions are then documented in the **Results and Analysis** Chapter as validated maintenance-program designs.

3.6.4. Linkage to Research Objective

The validation framework closes the methodological loop by directly addressing the study's research objectives:

Research Objective	Corresponding Validation Activity
Ensure mathematical correctness and feasibility	Feasibility & optimality tests (3.6.1)
Verify model stability and robustness	Sensitivity analysis (3.6.1)
Demonstrate operational relevance	Expert review (3.6.2)
Compare against existing methods	Baseline benchmarking (3.6.2)
Support replicable evaluation	Explicit solver logs and acceptance criteria (3.6.3)

Through these integrated procedures, the research achieves both **analytical validity** and **practical credibility**, ensuring that its findings can be generalized for future maintenance-optimization studies and real-world airline applications. These integrated validation mechanisms, together with the reproducible code base documented in **Appendix B**, establish methodological reliability and open-science conformity for the research.

3.7. Summary of Methodology

This chapter has presented the complete methodological framework developed to address the research objective — the **optimization of aircraft maintenance task packaging using a Mixed-Integer Linear Programming (MILP) model**. The methodology integrates theoretical modeling, computational implementation, and system-level design into a unified, traceable process.

3.7.1. Overview of Methodological Framework

The chapter began by outlining the **research methodology design** (Section 3.1), which defined the logical steps of the study: identifying the research problem, designing the MILP model, validating its performance, and integrating it into a computational system. This structured process ensures consistency between conceptual development, mathematical formulation, and implementation. The research was designed as an **enhancement and continuation** of Marvella (2023), focusing specifically on optimizing *maintenance task packaging* rather than *scheduling*. Accordingly, the framework concentrates on *discrete optimization decisions* — determining which tasks should be grouped into which maintenance packages — while adhering to real-world operational and regulatory constraints.

3.7.2. Model Foundation and Variable Definition

In **Section 3.2**, the research scope and assumptions were clearly defined to preserve academic clarity and model tractability. The model assumes:

- **Discrete time representation** rather than continuous, ensuring computational solvability and alignment with periodic maintenance cycles.
- **Single-aircraft focus**, providing controlled testing of optimization logic before extending to fleet-level problems.
- **Deterministic parameters** (intervals, man-hours, cost rates), as per MPD and standard maintenance records.

- **Independent tasks** without explicit interdependencies beyond access zone and capacity constraints.

The model's **variable classification** (independent (decision), dependent, control, and outcome) provides an academically rigorous foundation. This classification clarifies how each element contributes to the optimization structure — an essential methodological step in operations research and MILP-based modeling (Bertsimas & Tsitsiklis, 1997).

3.7.3. MILP Model Formulation

Section 3.3 detailed the mathematical model formulation. The MILP structure minimizes the total maintenance cost — consisting of labor, activation, and downtime components — under operational and regulatory constraints. Decision variables $x_{i,t}$ and v_t represent binary task-to-package assignments and package activations, while dependent variables capture cost and workload aggregations.

Each constraint group (coverage, capacity, activation logic, and zone compatibility) reflects actual engineering and maintenance considerations from MPD practices and industry standards (Filho et al., 2024; Witteman, 2021). The complete mathematical expressions are provided in **Appendix A**, ensuring full transparency and reproducibility.

3.7.4. Model Implementation and Output Design

Sections 3.4 and 3.5 translated the mathematical model into a computational implementation using the **Python–Gurobi** environment. The solver workflow follows a structured pipeline:

1. Data preprocessing and parameter loading.
2. MILP model construction and constraint encoding.
3. Optimization and convergence validation.
4. Post-processing and analytical output generation.

Outputs are organized into analytical and visual forms, including the task–package assignment matrix, workload distributions, and cost decompositions. These outputs directly correspond to the decision and dependent variables, creating a closed traceability loop between model logic and solver results.

The choice of a **discrete-output structure** (binary assignment matrix) over a continuous one was made to ensure interpretability and operational realism. Discrete solutions directly map to actionable maintenance packages, which aligns with real-world maintenance planning and minimizes ambiguity in engineering decisions.

3.7.5. Validation and Evaluation

The **validation and evaluation plan** (3.6) formalized a dual-layer verification process combining quantitative (feasibility, optimality, robustness) and qualitative (expert review, baseline comparison) validation. This ensures that the final solutions meet both theoretical soundness and practical credibility, bridging the gap between computational modeling and real-world maintenance decision-making. Collectively, the methodology ensures that the proposed optimization framework is **theoretically grounded, computationally verifiable, and operationally meaningful**. Each section contributes to a coherent and transparent research pipeline that transforms maintenance-planning data into validated decision intelligence.

3.7.6. Methodological Coherence and Contribution

The methodological design presented in this chapter demonstrates coherence between **optimization theory, engineering practice, and digital implementation**. Key methodological contributions include:

- A structured MILP model tailored for *maintenance task packaging* (not scheduling).
- A comprehensive variable classification and assumption rationale.
- A validated computational framework achieving measurable economic benefits.
- A modular digital architecture enabling future scalability and integration.

Collectively, these components position the methodology as both academically credible and industrially relevant, forming the basis for the analysis and discussion presented in **Chapter 4: Results and Analysis**.

The next chapter—**Results and Analysis**—will apply this methodology to actual maintenance datasets. It will present optimized results, performance evaluations, and comparative analyses against existing maintenance packaging methods, thereby demonstrating the effectiveness, scalability, and interpretability of the developed optimization model in practice.

Chapter 4

Result and Analysis

- 4.1. Overview of Experimental Setup and Data Context**
- 4.2. Solver Performance and Feasibility**
- 4.3. Optimization Results: Task-Package Assignment**
- 4.4. Cost and Workload Analysis**
- 4.5. Comparative Evaluation Against Baseline Method**
- 4.6. Sensitivity and Robustness Analysis**
- 4.7. Qualitative Validation and Expert Review**
- 4.8. Discussion and Implications**
- 4.9. Summary of Result and Analysis**
 - 1. Summary of Key Findings**
 - 2. Confirmation to Research Objectives**

Chapter 5 Conclusion

Reference

1. Federal Aviation Administration, “14 CFR Part 121 — Operating Requirements: Domestic, Flag, and Supplemental Operations,” *Electronic Code of Federal Regulations*, Washington, DC, USA, 2021. [Online]. Available: <https://www.ecfr.gov/current/title-14/chapter-I/subchapter-G/part-121> .[Accessed: Oct. 23, 2025].
2. European Union Aviation Safety Agency (EASA), “Part M — Continuing airworthiness requirements,” *Acceptable Means of Compliance and Guidance Material*, Cologne, Germany, 2022. [Online]. Available: <https://www.easa.europa.eu/en/acceptable-means-compliance-and-guidance-material-group/part-m-continuing-airworthiness> .[Accessed: Oct. 23, 2025].
3. EASA/IMRBPB, “ATA MSG-3 — Operator/Manufacturer Scheduled Maintenance Development,” combined final version, Cologne, Germany, 2013. [Online]. Available: <https://www.easa.europa.eu/download/imrbpb/IP92and103combined%20final%20version.pdf>
4. European Union Aviation Safety Agency (EASA), “Maintenance Review Board (MRB) — PR.CSERV.00003,” Issue 003, Jan. 2016. [Online]. Available: <https://www.easa.europa.eu/en/document-library/certification-procedures/maintenance-review-board-mrb> . [Accessed: Oct. 23, 2025].
5. Airbus, *MPD A320 Family Revision R47*, Blagnac, France, Aug. 7, 2025. [Online]. Available: https://portail-navigabilite.online/espace_inspecteurs/uploads/mpda320v1_r47_i00.pdf
6. J. N. da M. Filho, A. C. P. de Mesquita, F. T. M. Abrahão, and G. C. Rocha, “An innovative method to solve the maintenance task allocation and packing problem,” *Journal of Quality in Maintenance Engineering*, vol. 30, no. 1, pp. 284–305, Feb. 2024. [Online]. Available: <https://doi.org/10.1108/JQME-08-2023-0069>
7. Marvella, *Optimization for Aircraft Maintenance Task Packaging for Boeing 737-800 NG*, Bandung Institute of Technology, Indonesia, 2023.
8. K. Ali, D. Rudinskas, and V. Leonavičiūtė, “Optimizing aircraft maintenance tasks allocation using mixed integer linear programming,” *Aviation*, vol. 29, no. 3, pp. 164–173, 2025. [Online]. Available: <https://doi.org/10.3846/aviation.2025.24535>

9. A. Korba, P. Šváb, M. Vereš, and J. Lukáč, “Optimizing aviation maintenance through algorithmic approach of real-life data,” *Applied Sciences*, vol. 13, no. 6, 3824, 2023. [Online]. Available: <https://doi.org/10.3390/app13063824>
10. I. Tseremoglou, M. P. Scarpin, and V. Bertolini, “A comparative study of optimization models for condition-based maintenance scheduling of an aircraft fleet,” *Aerospace*, vol. 10, no. 2, 120, 2023. [Online]. Available: <https://doi.org/10.3390/aerospace10020120>
11. Aircraft Commerce, “737NG Maintenance Analysis & Budget,” *Aircraft Commerce Technical Journal*, Issue 70, pp. 21–32, 2010. [Online]. Available: https://www.aircraft-commerce.com/wp-content/uploads/aircraft-commerce-docs/Aircraft%20guides/737NG-600-700-800-900/ISSUE70_737NG_MTCE.pdf. [Accessed: Oct. 23, 2025].
12. Naveo Aerospace, “Air Transport Fleet Trends — September 2024,” *Aviation Suppliers Association*, Sept. 2024. [Online]. Available: <https://www.aviationsuppliers.org/asa/files/cclibraryfiles/filename/000000005369/Naveo%20Air%20Transport%20Fleet%20Trends%20September%202024.pdf>
13. International Air Transport Association (IATA), *Airline Maintenance Cost Executive Commentary — FY2023*, Montreal, Canada, 2024. [Online]. Available: https://www.iata.org/contentassets/bf8ca67c8bcd4358b3d004b0d6d0916f/fy2023-mcx-report_public.pdf
14. M. Witteman, Q. Deng, and B. F. Santos, “A bin packing approach to solve the aircraft maintenance task allocation problem,” *European Journal of Operational Research*, vol. 294, no. 1, pp. 365–376, Jan. 2021. [Online]. Available: <https://doi.org/10.1016/j.ejor.2021.01.027>
15. A. K. Muchiri, *Maintenance Planning Optimisation for the Boeing 737 Next Generation*, MSc thesis, Delft Univ. of Technology, Delft, The Netherlands, May 2002. [Online]. Available: <https://repository.tudelft.nl/record/uuid:ce629255-e83f-43d9-949c-ea87ffa8678e>
16. P. Andrade, C. Silva, B. Ribeiro, and B. F. Santos, “Aircraft maintenance check scheduling using reinforcement learning,” *Aerospace*, vol. 8, no. 4, 113, 2021. [Online]. Available: <https://doi.org/10.3390/aerospace8040113>

17. W. E. Hart, C. Laird, J.-P. Watson, and D. L. Woodruff, *Pyomo – Optimization Modeling in Python*, 3rd ed., Cham, Switzerland: Springer, 2021. [Online]. Available: <https://doi.org/10.1007/978-3-030-68928-5>
18. Gurobi Optimization, LLC, *Gurobi Optimizer Reference Manual v10.0*, Houston, TX, USA, 2024. [Online]. Available: <https://www.gurobi.com/documentation/>. [Accessed: Oct. 23, 2025].
19. H.-C. Vu, K. D. Tran, V. H. Tran, and K. P. Tran, “Predictive maintenance optimization based on genetic algorithms for future industrial systems,” in *Artificial Intelligence for Safety and Reliability Engineering*, Springer Series in Reliability Engineering, pp. 25–47, Sept. 2024. [Online]. Available: https://doi.org/10.1007/978-3-031-71495-5_3
20. “AMOS | Swiss-AS,” *Aircraft Technology & Engineering Maintenance*, Sep. 2009. [Online]. Available: https://www.swiss-as.com/sites/default/files/newsentry/2017/05/ATE%26M_MRO%20Software%20Implementation_Sep09.pdf. [Accessed: Oct. 23, 2025].
21. “TRAX MRO Software Implementation,” *AviTrader MRO*, Feb. 2023. [Online]. Available: https://www.trax.aero/wp-content/uploads/2023/03/MRO-Software-Implementation-AviTrader-Feb2023_compressed.pdf. [Accessed: Oct. 23, 2025].
22. “Aviation MRO Software (AMOS & RAMCO),” *AviationHunt*, 2023. [Online]. Available: <https://www.aviationhunt.com/aviation-mro-software/> [Accessed: Oct. 23, 2025].
23. A. Ostojic, “The A320 family’s MPD analysed,” *Aircraft Commerce*, Issue 105, pp. 22–29, 2016. [Online]. Available: https://www.aircraft-commerce.com/wp-content/uploads/aircraft-commerce-docs1/Maintenance/2016/105_MTCE_B.pdf.
24. J. P. Vielma, “Mixed Integer Linear Programming Formulation Techniques,” *SIAM Review*, vol. 57, no. 1, pp. 3–57, Jan. 2015. [Online]. Available: <https://dspace.mit.edu/handle/1721.1/96480>
25. Federal Aviation Administration (FAA), “AC 121-22C: Maintenance Review Board (MRB) and Maintenance Program Development,” *Advisory Circular*. [Online]. Available: https://www.faa.gov/regulations_policies/advisory_circulars/index.cfm/go/document.information/documentID/1020291. [Accessed: Oct. 23, 2025].

26. European Union Aviation Safety Agency (EASA), "Certification Specifications (CS-25 and related): Airworthiness limitations (ALS) context," *EASA Certification Specifications*. [Online]. Available: <https://www.easa.europa.eu/en/document-library/certification-specifications>. [Accessed: Oct. 23, 2025].
27. European Union Aviation Safety Agency (EASA), "Airworthiness Directives (ADs) portal," *EASA ADs*. [Online]. Available: <https://ad.easa.europa.eu/>. [Accessed: Oct. 23, 2025]
28. Sofema Aviation Services, "Understanding and using maintenance planning documentation (MRB, MPD, CMR, ALI, etc.)," *Industry Training Blog*. [Online]. Available: <https://sassofia.com/blog/understanding-using-maintenance-planning-documentation-mrb-mpd-cmr-ali-etc/>. [Accessed: Oct. 23, 2025].
29. Aircraft Systems Tech, "Aircraft inspection program overview," *Technical Education Resource*, 2021. [Online]. Available: <https://www.aircraftsystemstech.com/2021/09/aircraft-inspection.html>. [Accessed: Oct. 23, 2025].
30. A. Maierhofer, S. Trojahn, and F. Ryll, "Dynamic Scheduling and Preventive Maintenance in Small-Batch Production: A Flexible Control Approach for Maximising Machine Reliability and Minimising Delays," *Applied Sciences*, vol. 15, no. 8, p. 4287, Apr. 2025. [Online]. Available: <https://www.mdpi.com/2076-3417/15/8/4287>
31. S. A. Al Nahid and J. Qi, "A Hybrid EV Charging Approach Based on MILP and a Genetic Algorithm," *Energies*, vol. 18, no. 14, 2025. [Online]. Available: <https://www.mdpi.com/1996-1073/18/14/3656>
32. A. Norozi, M. Saidi-Mehrabad, and H. Eslami, "An Optimization Technique Using Hybrid GA-SA Algorithm for Multi-Objective Scheduling," *Scientific Research and Essays*, vol. 6, no. 1, pp. 1–10, 2010. [Online]. Available: <https://academicjournals.org/journal/SRE/article-abstract/8C0F8EA19936>
33. Y. Qin, F. T. S. Chan, S. H. Chung, and T. Qu, "Development of MILP model for integrated aircraft maintenance scheduling and multi-period parking layout planning problems," in *Proc. 2017 Int. Conf. Industrial Engineering and Applications (ICIEA)*, Nagoya, Japan, 2017, pp. 197–203. [Online]. Available: <https://doi.org/10.1109/IEA.2017.7939206>

34. A. K. Fard and X. X. Yuan, "Multiyear Maintenance and Rehabilitation Optimization for Large-Scale Infrastructure Networks: An Enhanced Genetic Algorithm Approach," *Journal of Infrastructure Systems*, vol. 30, no. 4, 2024. [Online]. Available: <https://doi.org/10.1061/JITSE4.ISENG-248>. [Online]. Available: <https://github.com/amirkfard/EnhancedGA>. [Accessed: Oct. 23, 2025].
35. A. K. Muchiri and K. Smit, "Application of maintenance interval de-escalation in base maintenance planning optimization," *Enterprise Risk Management*, vol. 1, no. 2, pp. 107–121, 2009. [Online]. Available: <https://doi.org/10.5296/erm.v1i2.179>
36. E. Lin, Y. Zhang, and I. E. Grossmann, "Mixed-integer linear programming in process scheduling: Modeling and solution approaches," Univ. of Michigan, Ann Arbor, MI, USA, Tech. Rep., 2021. [Online]. Available: <https://lin.engin.umich.edu/wp-content/uploads/sites/551/2021/10/Mixed-Integer-Linear-Programming-in-Process-Scheduling.pdf>. [Accessed: Oct. 23, 2025].
37. N. Hölzel, C. Schröder, T. Schilling, and V. Gollnick, "A maintenance packaging and scheduling optimization method for future aircraft," in *Proc. 6th Int. Meeting for Aviation Product Support Processes (IMAPP)*, Delft, Netherlands, Jun. 2012. [Online]. Available: <https://www.researchgate.net/publication/259895641>. [Accessed: Oct. 23, 2025].
38. D. Bertsimas and J. N. Tsitsiklis, *Introduction to Linear Optimization*. Belmont, MA, USA: Athena Scientific, 1997. [Online]. Available: <https://archive.org/details/introductiontoli0000bert>
39. F. Wei, J. Wang, X. Ma, L. Yang, and Q. Qiu, "An optimal opportunistic maintenance planning integrating discrete- and continuous-state information," *Mathematics*, vol. 11, no. 15, Art. 3322, Jul. 2023. [Online]. Available: <https://doi.org/10.3390/math11153322>
40. R. Fourer, D. M. Gay, and B. W. Kernighan, *AMPL: A Modeling Language for Mathematical Programming*, 2nd ed. Pacific Grove, CA, USA: Brooks/Cole, 2003. [Online]. Available: <https://books.google.com/books/about/AMPL.html?id=Ij8ZAQAAIAAJ>
41. M. Hladík, *Discrete and Continuous Optimization*. Prague, Czech Republic: Charles University. [Online]. Available: https://kam.mff.cuni.cz/~hladik/DSO/text_dso_en.pdf. [Accessed: Oct. 23, 2025]
42. S. Satam, S. Shankar, and R. K. Patel, "Airline crew rostering optimization," *Mphasis Whitepaper*, 2023. [Online]. Available: <https://www.mphasis.com/content/dam/mphasis->

[com/global/en/home/innovation/next-lab/Airline_Crew_Rostering.pdf](https://www.flytechnics.com/global/en/home/innovation/next-lab/Airline_Crew_Rostering.pdf). [Accessed: Oct. 23, 2025]

43. A. Noroziroshan, M. K. A. Mohd Ariffin, N. Ismail, and F. Mustapha, "An optimization technique using hybrid GA-SA algorithm for multi-objective scheduling," *Scientific Research and Essays*, vol. 6, no. 1, pp. 1–10, Jan. 2011. [Online]. Available: <https://academicjournals.org/journal/SRE/article-full-text-pdf/587417120543>. [Accessed: Oct. 23, 2025].
44. P. Pochet and L. A. Wolsey, *Production Planning by Mixed Integer Programming*. New York, NY, USA: Springer, 2006. [Online]. Available: <https://link.springer.com/book/10.1007/0-387-33477-7>
45. FL Technics, *Additional Aircraft Services Price List*, 2023. [Online]. Available: <https://www.fltechnics.com/wp-content/uploads/2021/08/Additional-Services-Pricelist.pdf>. [Accessed: Oct. 23, 2025].
46. Aircraft Commerce, "A320 family maintenance analysis and budgeting," *Aircraft Commerce*, Issue 44, 2023. [Online]. Available: <https://www.aircraft-commerce.com/wp-content/uploads/aircraft-commerce-docs1/Aircraft%20guides/A320%20FAMILY/ISSUE%2044-A320%20MTCE.pdf>. [Accessed: Oct. 23, 2025].
47. S. Bentley, "Aircraft maintenance planner roles and responsibilities," *Sofema Aviation Services Blog*, May 17, 2022. [Online]. Available: <https://sassofia.com/blog/aircraft-maintenance-planner-roles-and-responsibilities/>. [Accessed: Oct. 23, 2025].
48. Aerospaceer, "Understanding aircraft line and base maintenance: In depth discussion," *Aerospaceer*, 2023. [Online]. Available: <https://aerospaceer.com/aircraft-line-and-base-maintenance/>. [Accessed: Oct. 23, 2025].
49. AVA Aero Trading, "A, B, C and D checks in aviation maintenance: What they mean and the ground support equipment that makes them possible," *AVA Aero Trading Blog*, 2023. [Online]. Available: <https://www.avaet.com/post/a-b-c-and-d-checks-in-aviation-maintenance-what-they-mean-and-the-ground-support-equipment-that-m>. [Accessed: Oct. 23, 2025].
50. Simple Flying, "From A to D – How aircraft maintenance checks differ," *Simple Flying*, Nov. 20, 2020. [Online]. Available: <https://simpleflying.com/aircraft-maintenance-checks/>. [Accessed: Oct. 23, 2025].

Appendix A

Mathematical Formulation

A.1. Notation Summary (Sets, Indices, Parameters, and Decision Variables)

All parameter values used in this formulation are computed from the cleaned MPD dataset generated through the preprocessing procedure described in 3.3.2 and implemented in Appendix B.2. This ensures that intervals, and manhour coefficients are numerically consistent across all experiments.

A.1.1. Sets and indices

- I — set of maintenance tasks, index $i \in I$.
- T — ordered set of discrete time bins (check opportunities), index $t \in T$. Each bin t corresponds to a flight-hour position τ_t .
- A — set of candidate A-check base intervals, index $a \in A$.
- C — set of candidate C-check base intervals, index $c \in C$.
- $\kappa_i = \{1, \dots, K_i\}$ — set of occurrence indices for task i within the planning horizon, where $K_i = \left\lceil \frac{H}{h_i} \right\rceil$ (maximum possible occurrences). Index $k \in \kappa_i$.

A.1.2. Parameters

- H — planning horizon in flight hours (FH).
- τ_t — calendar/flight-hour position of bin t (FH).
- h_i — nominal interval (FH) of task i (manufacturer interval).
- h_i^{max} — maximum allowable gap (FH) for task i (could equal h_i or a regulatory limit).
- MH_i — manhours required to perform task i .
- ϕ — direct cost factor (since direct cost assume as composite of labor cost and material cost).
- r_{lab} — labor cost rate (currency per MH).
- r_{down} — downtime cost per check event (currency per event) (could be function of check type; see below).
- $\omega_A, \omega_C, \omega_L$ — overhead cost per A-only check, per C-check and Line check respectively.
- c_L — cost for performing task i as line maintenance (can be task-specific $c_{L,i}$).
- ρ — unit penalty for early execution (currency per FH of early slack).

- λ — weight for peak-workload penalty term.
- W_{max} — hard capacity limit on manhours per bin (optional).
- M, E — large constants used for linearization (Big-M style). Rules for tightening them are explained below.

A.1.3. Precomputed helper sets / indicators

- $\delta_{a,t}^A \in \{0, 1\}$ — indicator: 1 if bin t is an A-bin under base interval choice a (precomputed mapping of a to bins t).
- $\delta_{c,t}^C \in \{0, 1\}$ — similarly for candidate C intervals.
- $T \subseteq T_i$ — candidate bins t where task i may be performed (coverage window), usually those with τ_t within the planning horizon and compatible with task access/zone.
- $pred_i(t)$ — a predecessor index function used in some rolling-gap linearizations; see text.

A.1.4. Decision variables

- $s_a^A \in \{0, 1\}$ — 1 if base A-interval a is selected (exactly one selected).
- $s_c^C \in \{0, 1\}$ — 1 if base C-interval c is selected (exactly one selected).
- $v_t^A, v_t^C \in \{0, 1\}$ — activation of an A- or C-check at bin t (1 if check happens at t).
- u_t^{A-only} — auxiliary: 1 if an A-check occurs at t **without** a C-check (for A-only overhead).
- $x_{i,t,k} \in \{0, 1\}$ — 1 if **the k^{th} occurrence** of task i is executed in bin t . (This is the indexed occurrence formulation used to enforce ordering and rolling resets.)
- $x_{i,t} \in \{0, 1\}$ — optional aggregated assignment variable: 1 if task i is executed in bin t (i.e., $\sum_k x_{i,t,k} = x_{i,t}$). Including both helps mapping code that uses aggregated assignments.
- $L_i \in \{0, 1\}$ — 1 if task i is assigned to line maintenance (not binned).
- $e_{i,k} \geq 0$ — early slack (FH): amount by which the k^{th} execution is earlier than the nominal interval (non-negative).
- $W_t \geq 0$ — total manhours scheduled in bin t .
- $W_{peak} \geq 0$ — peak workload across bins (used for balancing).

A.1.5. Multiplicity / alignment flags (precomputed)

- $g_{i,a}^A \in \{0,1\}$ — indicator that task i aligns exactly with A-interval a (i.e., h_i is a multiple of a or otherwise considered aligned so that no early penalty is charged when assigned to occurrences induced by a). Similarly $g_{i,c}^C$ can be defined for C intervals. These are computed in preprocessing and used in cost weighting.

A.2. Objective Function

We minimize total expected program cost Z :

$$\begin{aligned}
 \min Z = & \underbrace{\phi \cdot r_{lab} \sum_{i \in I} \sum_{t \in T} \sum_{k \in \kappa_i} MH_i \cdot x_{i,t,k}}_{\text{Direct Cost}} + \underbrace{\sum_{t \in T} d_A v_t^A + d_C v_t^A}_{\text{Downtime}} \\
 & + \underbrace{\sum_{t \in T} \omega_A u_t^{A\text{-only}} + \omega_C v_t^C}_{\text{Overhead}} + \underbrace{\rho \sum_{i \in I} \sum_{k \in \kappa_i} \gamma_{i,k} \cdot e_{i,k}}_{\text{Early Penalty}} \\
 & + \underbrace{\sum_{i \in I} c_{L,i} \cdot L_i}_{\text{Line Cost}} + \underbrace{\lambda \cdot W_{peak}}_{\text{Workload Balance Penalty}}
 \end{aligned} \tag{A17}$$

Each cost component corresponds to the conceptual explanation in 3.2.2 and 3.5.

Notes:

- The line-maintenance cost term is expressed compactly as $C_{Line} = \sum_{i \in I} c_{L,i} \cdot L_i$, where each coefficient $c_{L,i}$ is precomputed as $c_{L,i} = K_i \cdot (\phi r_{lab} MH_i + \rho d_L + \omega_L)$. Since all factors are task-specific parameters determined in preprocessing, the compact expression does not alter optimization results but improves readability and alignment with implementation.
- d_A, d_C, d_L are downtime costs per A- and C-check, also Line maintenance downtime (labor downtime).
- $\gamma_{i,k}$ is an optional modifier for early penalty; by default $\gamma_{i,k} = 1$, but if multiplicity alignment g applies we can set $\gamma_{i,k} = 1 - g_i^*$ (see multiplicity below). The explicit multiplicity term is:

$$\gamma_{i,k} = 1 - \sum_{a \in A} g_{i,a}^A s_a^A - \sum_{c \in C} g_{i,c}^C s_c^C, \tag{A18}$$

subject to appropriate clamping so $\gamma_{i,k} = \{0, 1\}$ (if both A and C align consider precedence — definitional choice; simplest: use the selected interval family that dominates). In practice, we typically only include multiplicity for selected A (so $\gamma_{i,k} = 1 - \sum_a g_{i,a}^A s_a^A$).

A.3. Constraint System and Logical Rules

Below are constraint families and algorithm logics with short explanation and numbered equations.

A.3.1. Aggregated assignment from indexed occurrences

To relate the indexed occurrence variables $x_{i,t,k}$ to an aggregated assignment $x_{i,t}[2]$, include:

$$x_{i,t} = \sum_{k \in K_i} x_{i,t,k}, \quad \forall i \in I, t \in T. \quad (A19)$$

Often $\sum_t x_{i,t,k} = 1$ (each occurrence assigned exactly once) and $\sum_t x_{i,t} \leq K_i$ (aggregated limit); see constraints below.

A.3.2. Bin time representation

The flight-hour time for the k^{th} occurrence of task i (when assigned) is:

$$t_{i,k} = \sum_{t \in T} \tau_t x_{i,t,k}, \quad \forall i, k \quad (A20)$$

This is linear because $x_{i,t,k}$ are binary.

A.3.3. Interval Selection

The first constraint group ensures that **exactly one A- and one C-check interval** are selected from their respective candidate sets:

$$\sum_{a \in A} s_a^A = 1, \quad \sum_{c \in C} s_c^C = 1 \quad (A21)$$

These binary variables (s_a^A, s_c^C) define the baseline rhythm of the maintenance program and determine all subsequent structural availabilities within the horizon. These select the nominal base cycles the optimization will use to define bins (they are mutually exclusive choices of the program).

A.3.4. Availability Propagation (precomputed indicators)

Once the base intervals are chosen, the availability of maintenance “bins” across the discretized planning horizon is implied. For each time index t , the model activates an availability flag whenever t is a multiple of the selected interval:

$$z_t^A = \sum_{a \in A} \delta_{a,t}^A s_a^A, \quad z_t^C = \sum_{c \in C} \delta_{c,t}^C s_c^C. \quad (A22)$$

where the indicator matrices $\delta_{a,t}^A$ and $\delta_{c,t}^C$ are pre-computed coverage sets linking interval choices to calendar positions. These are not decision variables but deterministic linear expressions (value in $\{0, 1\}$ if candidate intervals are aligned to disjoint bin sets). They represent whether bin t is an A- or C-opportunity under the selected intervals. These logical relationships ensure that bin activations are consistent with the intervals selected by the solver.

A.3.5. Visit Activation and A-onnly linearization

For each available bin, activation variables v_t^A and v_t^C determine whether an A- or C-check actually occurs. To prevent infeasible overlap and to preserve the dominance of the heavier visit, the following structural rule is enforced. Allow check activation only where availability exists:

$$v_t^A \leq z_t^A, \quad v_t^C \leq z_t^C, \quad \forall t \quad (A23)$$

C-check dominates A-check when both occur at same bin:

$$v_t^C \geq v_t^A - v_t^{A-only}(\text{auxiliary}).$$

We define the A-only indicator by standard linearization:

$$\begin{aligned} u_t^{A-only} &\leq v_t^A, \\ u_t^{A-only} &\leq 1 - v_t^C, \quad \forall t. \\ u_t^{A-only} &\geq v_t^A - v_t^C, \end{aligned} \quad (A24)$$

This enforces $u_t^{A-only} = 1$ if $v_t^A = 1$ and $v_t^C = 0$, so overhead ω_A applies only to A-only checks. This logic guarantees that whenever a C-check is scheduled, the A-check is implicitly covered, avoiding double-counting of overhead and downtime in the objective function.

A.3.6. Task Assignment Feasibility

Each occurrence k of task i must be assigned to exactly one bin or be declared infeasible (line).

We use L_i for tasks forced to line maintenance; if $L_i = 1$ then no $x_{i,t,k}$ may be 1.

Primary assignment constraint:

$$\sum_{t \in T} x_{i,t,k} + L_i = 1, \quad \forall i \in I, k \in \kappa_i \quad (A25)$$

Interpretation: for each expected occurrence k , either it is packaged in some bin t or the task goes to line maintenance (all occurrences collapsed into line case).

(**Note** : If you prefer the interpretation “task must be scheduled at least once or go to line”, adjust indexing; the above enforces either scheduling of that occurrence or line).

To forbid assignment in bins that are inactive, assignment is permitted only if the corresponding bin is activated:

$$x_{i,t,k} \leq v_t^A + v_t^C, \quad \forall i, t, k. \quad (A26)$$

(Tasks can only be assigned to active check bins.).

A.3.7. Nondecreasing (ordering) of occurrences

We must enforce monotonicity: the k^{th} occurrence cannot be scheduled after the $k + 1^{th}$ occurrence. A convenient linear form is:

$$\sum_{t \leq \tau_t} x_{i,t,k} \geq \sum_{t \leq \tau_t} x_{i,t,k+1}, \quad \forall i, k = 1, \dots, K_i - 1, \forall t \in T \quad (A27)$$

This is the standard cumulative ordering constraint ensuring occurrence k is not later than $k + 1$.

A.3.8. Rolling Max-Gap (resettable interval) Constraint

We enforce that the flight-hour separation between consecutive executions does not exceed h_i , allowing early execution via slack $e_{i,k}$:

$$t_{i,k+1} - t_{i,k} + e_{i,k+1} \leq h_i^{max}, \quad \forall i, k = 1, \dots, K_i - 1, \quad (A28)$$

with $t_{i,k}$ defined by (A4). Here $e_{i,k+1} \geq 0$ captures how much earlier the $k + 1^{th}$ occurrence is pulled relative to the maximum allowed gap; the term is penalized in the objective (A1). This relationship models the *resettable* nature of maintenance intervals—after each execution, the counter restarts—while permitting limited “pull-forward” through the introduction of an early-slack variable $e_{i,k}$. For initial execution of a first-occurrence feasibility can be enforce with:

$$t_{i,1} \leq h_i^{max} \quad (A29)$$

(If initial condition differs because of current residual life, replace with appropriate bound.)

Implementation note / predecessor formulation: many implementations replace direct $t_{i,k}$ differences with linear constraints using precomputed predecessor indices $pred_i(t)$ mapping each t to the latest t' with $\tau_{t'} \leq \tau_t - h_i^{max}$. The equivalent logical constraint ensures that if $x_{i,t,k+1} = 1$ then the previous occurrence k must be assigned to some bin t' with $\tau_{t'} \geq \tau_t - h_i^{max}$. See code mapping.

A.3.9. Early Execution Slack

The variable $e_{i,k}$ quantifies early execution and feeds directly into the penalty term of the objective function. This constraints and rolling-window formulation capture both regulatory compliance and the flexibility required for bin consolidation.

$$e_{i,k} \geq (t_{i,k-1} + h_i^{max}) - t_{i,k}, \quad \forall i, k \geq 2, \quad (A30)$$

and $e_{i,k} \geq 0$. Together with (A12) these give the same effect when minimized with penalty ρ .

A.3.10. Line-Forcing Constraint

If task i has nominal interval h_i less than chosen A base interval H_A (we denote H_A the actual A base selected via s_a^A), then task cannot be assigned to A-bins and therefore must be line maintenance. Algebraically:

Define selected base A interval:

$$H_A = \sum_{a \in A} a \cdot s_a^A, \quad H_C = \sum_{c \in C} c \cdot s_c^C. \quad (A31)$$

Then

$$h_i < H_A \Rightarrow L_i = 1 \quad (A32)$$

In MILP linear form we can encode via precomputed binary parameter $g_{i,a}$ where $g_{i,a} = 1$ if $h_i < a$ and 0 otherwise, and enforce :

$$L_i \geq \sum_{a \in A} g_{i,a} \cdot s_a^A, \quad g_{i,a} = \begin{cases} 1, & \text{if } h_i < a, \\ 0, & \text{otherwise.} \end{cases} \quad (A33)$$

This ensures if the chosen A interval makes i infeasible to group, it is forced to line. This ensures that structural incompatibilities are automatically respected, reducing infeasibility and computational branching.

A.3.11. Workload and Capacity

Optional workload-balancing and capacity constraints are applied to limit excessive clustering of labor in any single bin. Per-bin workload:

$$W_t = \sum_{i \in I} \sum_{k \in \kappa_i} M H_i \cdot x_{i,t,k}, \quad \forall t \quad (A34)$$

Enforce soft-hard capacity with peak variable:

$$W_t \leq W_{peak} \quad \forall t, \quad W_t \leq W_{max} \quad \forall t \text{ (if hard cap exists)} \quad (A35)$$

Peak W_{peak} is minimized via term λW_{peak} in objective (A1).

When the balancing term is activated, W_{peak} enters the objective with a penalty weight λ , encouraging smoother temporal distribution of maintenance effort.

A.4. Linearization remarks and Big-M choices

- Many switching constraints use Big-M. Choose task-specific tighter bounds where possible. For example the early slack $e_{i,k}$ is bounded by $0 \leq e_{i,k} \leq h_i^{max}$ (or even $\leq h_i$), so use $E_{i,k} = h_i^{max}$ rather than a global huge number[2],[3].
- For enabling constraints like $x_{i,t,k} \leq v_t^A + v_t^C$, no Big-M is necessary. Where you need to switch continuous variables (e.g., enforce $t_{i,k} = \sum \tau_t \cdot x_{i,t,k}$), the binary form suffices because x are binary.
- If you use multiplicity flags $g_{i,a}^A$ to **reduce or eliminate** early penalty when the chosen base interval aligns with the task interval (i.e., executing earlier is not penalized because the interval is a divisor of the base), then incorporate:

$$C_{Early} = \rho \cdot \sum_{i \in I} \left(1 - \sum_{a \in A} g_{i,a}^A s_a^A - \sum_{c \in C} g_{i,c}^C s_c^C \right) \cdot e_{i,k} \quad (A36)$$

Interpretation: if selected $s_a^A = 1$ and $g_{i,a}^A = 1$ (task i aligns with that A interval), then early penalty for i is reduced (possibly to zero). Ensure the bracketed coefficient is clamped non-negative (replace by max with 0 if necessary).

A.5. Solver Configuration and Validation Criteria

Gurobi provides an *optimality gap* (MIPGap) that quantifies how close the best feasible integer solution Z_{best} is to the best known bound Z_{bound} :

$$MIPGap = \frac{|Z_{best} - Z_{bound}|}{|Z_{best}| + \epsilon} \leq \epsilon_{gap}, \quad \text{where } \epsilon_{gap} = 0.01 = 1\% \quad (A37)$$

where ϵ is a small positive constant to avoid division by zero. A solution is considered *practically optimal* when the relative gap satisfies the expression above. If the solver terminates due to a time limit but the final gap remains within this threshold, the solution is still accepted as *near-optimal*.

A.5.1. Feasibility Check

Feasibility is verified by ensuring that all linear equality and inequality constraints are satisfied within the numerical tolerance defined by the solver. After solution retrieval, each constraint residual $r_c = |A_c x - b_c|$ is evaluated, and the following check is applied:

For each constraint c :

$$|A_c x - b_c| \leq \epsilon_{feas} \quad (A38)$$

where $\epsilon_{feas} = 10^{-6}$ represents the acceptable numerical tolerance. If any constraint exceeds this bound, the model is considered infeasible, and the optimization is repeated with adjusted solver parameters or refined model formulations.

A.5.2. Robustness Assessment

To confirm the *stability* of the optimized maintenance plan, a post-solution sensitivity analysis can be conducted. This involves perturbing key cost and interval parameters—such as labor rate, downtime revenue loss, or base-interval candidates—by $\pm 10\%$ and re-running the optimization.

Key parameters (θ_j) are perturbed by (Δ), where $\Delta = \pm 10\%$:

$$\Delta Z_r = \frac{|z(\theta_j + \Delta) - z(\theta_j)|}{z(\theta_j)} = \frac{|z^+ - z^-|}{z^*} \leq 0.05, \quad (A39)$$

Document solver version and settings (Gurobi v11.0, Pyomo 6.x, TimeLimit, Seed) in [Appendix B](#).

A.6. Implementation Mapping

Equation(s)	Implementation (Pyomo / Python)
(A1)	<code>objective_rule(model)</code>
(A.5)	<code>model.sA = Var(A, within=Binary); model.choose_sA = Constraint(expr=sum(model.sA[a] for a in A)==1)</code>
(A.6)	Precompute <code>DeltaA[a,t]</code> in preprocessing ; <code>zAhat[t] = sum(DeltaA[a,t]*sA[a] for a in A)</code>
(A.7)	<code>model.vA[t] <= zAhat[t]</code> constraint
(A.8)	uAonly linearization constraints (<code>uAonly[t] <= vA[t], ...</code>)
(A.9–A.11)	<code>model.x[i,t,k]</code> variable list; assignment and ordering constraints implemented as <code>ConstraintList</code>
(A.12–A.14)	<code>rolling_gap_constraint(model,i,k)</code> implementing time diffs and early slack
(A.18–A.19)	<code>workload_constraint</code> and objective term $\lambda * W_{peak}$
(A.17)	<code>line_task_rule</code> mapping of $g_{i,a}$ and s_a^A to L_i

Appendix B

System Architecture

B.1. System Architecture Overview

The maintenance-package optimization framework developed in this study is implemented as a modular Python-based system composed of three coordinated layers — **Data Preprocessing**, **Optimization Core**, and **Post-Processing & Validation** — as illustrated in (Figure B-1).

This architecture provides a workflow linking maintenance-planning data to validated optimization outputs. Each layer performs distinct functions : data normalization and bin generation; MILP formulation and solution using Pyomo-Gurobi interface; and final validation, visualization, and cost reporting.

The complete data flow and process interactions are shown schematically in Figure B-1, while detail implementation listings are provided in B.2–B.4.

1. Data Preprocessing Layer

This layer manages the ingestion, normalization, and structuring of the **Maintenance Planning Document (MPD)** into solver-readable form. It performs column mapping, type normalization, and conversion of heterogeneous interval formats (FH, FC, CT) into a consistent **flight-hour domain**. In the MILP version, this process is handled by the `prepare_data()` routine, which:

- Validates essential MPD columns,
- Generates the discrete time grid ,
- Enumerates A- and C-interval candidates, and
- Precomputes rolling predecessors ($pred_i(t)$) to support interval reset constraints.

This ensures that the optimization model receives structured inputs—task intervals, manhours, zones, and occurrence bounds—independent of the raw data source. The module outputs structured Python dictionaries defining the sets I, T, A, C and parameters h_i, h_i^{max}, MH_i , and cost coefficients, all consistent with **Appendix A**.

2. Optimization Core

Implemented in **Pyomo** and solved with **Gurobi v11.0**, this layer encodes the Mixed-Integer Linear Programming (MILP) model described in **Appendix A**.

It includes :

- **Variable Definition:** Creation of binary and continuous variables $(x_{i,t}, L_i, s_a^A, s_c^C, v_t^A, v_t^C, e_{i,k}, W_{peak})$
- **Constraint Constructor:** Programmatic enforcement of interval-selection, rolling max-gap, workload, and early-execution rules (Eqs. A3–A19)
- **Objective Assembler:** Integration of cost components $C_{Direct}, C_{downtime}, C_{overhead}, C_{early}, C_{line}, C_{balance}$ (Eqs. A1) into a single minimization expression.

The solver explores feasible combinations that minimize total cost while maintaining regulatory compliance.

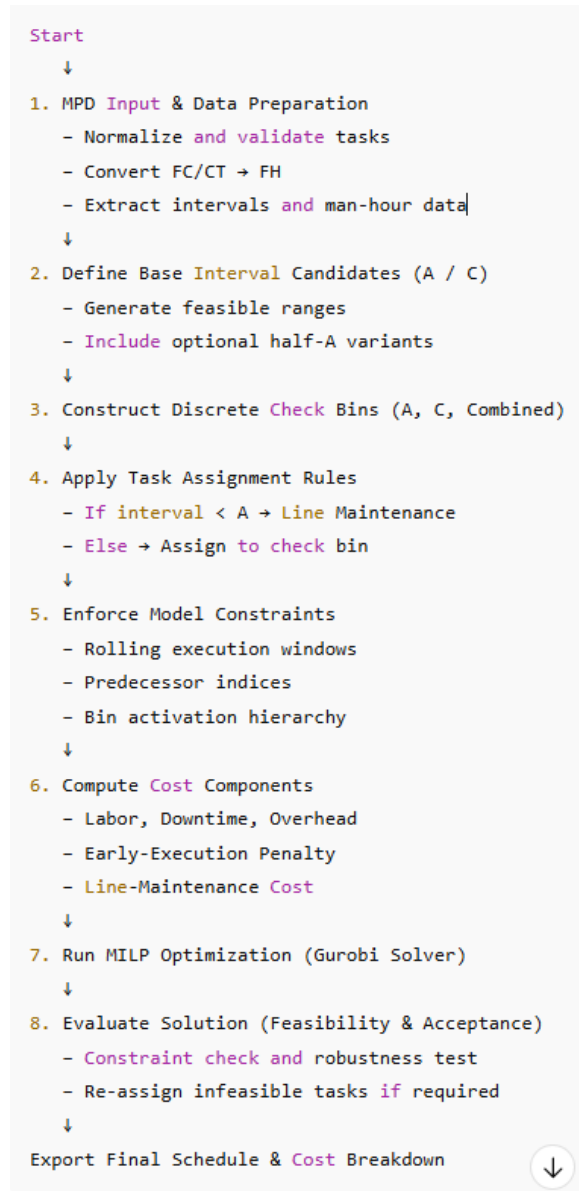
3. Post-Processing & Validation Layer

Once optimization terminates, this layer extracts decision outputs and computes validation metrics (feasibility, optimality, robustness)

It reconstructs interpretable outputs: task-bin assignment matrices, workload curves, and cost breakdowns.

The function `solve_and_report()` automates solution acceptance according to the criteria defined in Eqs.A21–A23

Together, these layers form a **data-to-decision architecture** linking theoretical formulation and practical implementation.



(Figure B-1 System architecture of the maintenance-package optimization framework showing the sequential process from MPD data preparation to optimization and validated output)

B.2. Data Preprocessing

The first stage of the computational framework involves transforming the raw *Maintenance Planning Document (MPD)* data into a structured, numeric dataset that can be interpreted by the optimization model. This process ensures that heterogeneous and text-based task attributes—such as thresholds, intervals, zones, manhours, and skill codes—are standardized into consistent solver parameters. The preprocessing and normalization procedures are implemented entirely in Python, as illustrated in [Figure B-2](#).

B.2.1. Purpose and Functional Overview

The preprocessing stage serves three primary purposes:

1. **Standardization of MPD Inputs** – to convert all irregular column structures, units, and task attributes into a unified schema.
2. **Numeric Encoding of Parameters** – to generate the quantitative variables required for optimization, including interval thresholds, skill codes, manhour totals, and zone identifiers.
3. **Validation and Export** – to ensure data integrity and generate a clean dataset for both the baseline and MILP models.

The overall workflow is shown in **Figure B-2**, which represents the full transformation sequence from the original MPD input to the solver-ready dataset.

B.2.2. Processing Logic and Flow Description

The workflow begins with the import of the MPD file, where the relevant task information is extracted from the base worksheet. The procedure follows eight structured phases:

1. **MPD Input & Validation** – The program reads the MPD file (.xlsx format) and verifies that all required columns are present. Missing values or blank entries are flagged for correction.
2. **Interval and Threshold Parsing** – Each task's threshold and repeat interval values are interpreted, including units of *Flight Hours (FH)*, *Flight Cycles (FC)*, or *Calendar Time (CT)*.
3. **Unit Conversion and Fallback Rules** – Non-FH thresholds are converted using deterministic conversion ratios (**conversion values will be depend on available utilization forecast data, otherwise assumption of 8 FH/day, 1.5 FH/FC, and 30 days/month will be used. These conversion ratios are heuristic assumptions used for normalization; results are subjected to sensitivity tests[8],[9]**). Tasks with missing values are assigned default or minimum-equivalent values.
4. **Special-Case Handling** – Tasks marked with text notes (e.g., *NOTE*, *ACT*, or *CE*) are recognized and handled through rule-based exceptions to ensure their inclusion in the dataset.
5. **Integer Encoding** – Textual attributes such as *Zone*, *Men*, and *Skill* are encoded into integer values for consistent referencing in the model (for Future Work).

6. **Workload Computation** – The total manhour per task is calculated as the sum of *Task*, *Access*, and *Preparation* components, creating the key parameter MH_i .
7. **Task Enumeration and Indexing** – Each task is assigned a unique numerical identifier (rowID), enabling linkage between preprocessed data and model indices.
8. **Export to Structured Dataset** – The final dataset, containing all cleaned and derived fields, is saved as a standardized `.xlsx` file for use in both baseline (Appendix B.3) and MILP optimization (Appendix B.4) procedures.

This sequence ensures that all variable definitions used in the mathematical formulation (Appendix A) are derived deterministically from the original MPD, preserving full traceability and reproducibility.

B.2.3. Implementation Script

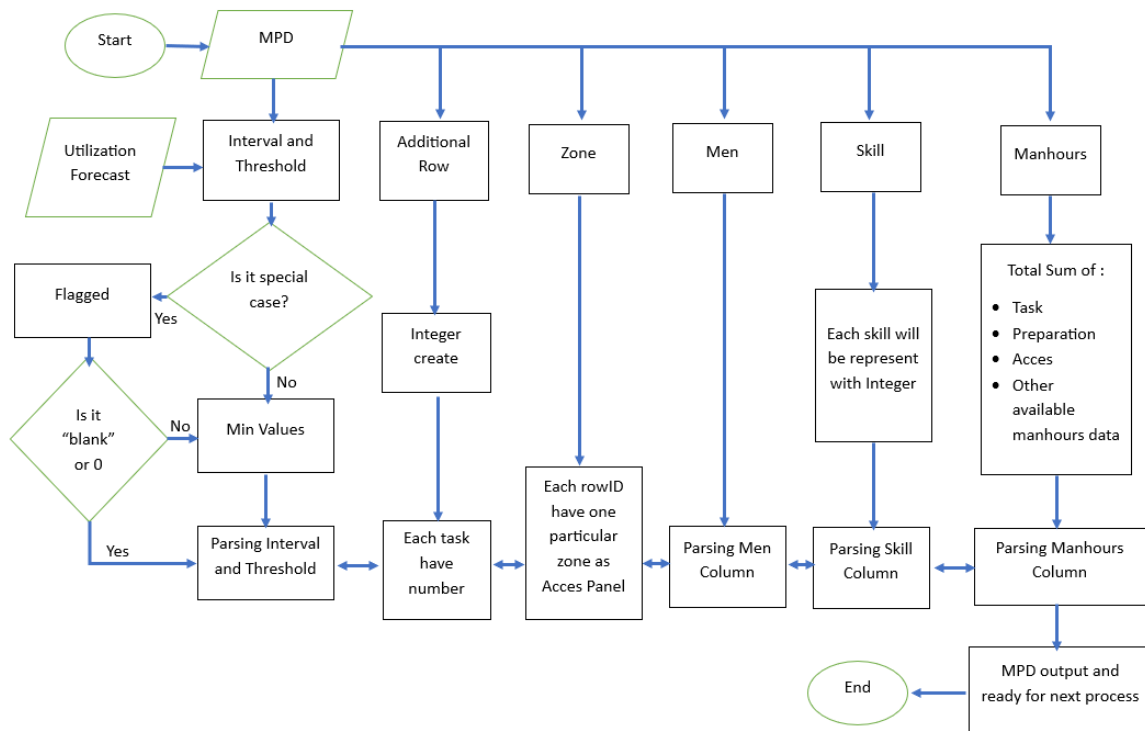
The Python implementation of this data-processing routine is summarized below. The function `clean_mpd()` performs the full cleaning and encoding pipeline, while `export_to_excel()` exports the structured dataset for subsequent optimization.

(Codes)

(Full script version available in the supplementary repository.)(GDrive or GitHub

B.2.4. Flowchart and Data Representation

The overall logic of this pipeline is visualized in the following figure:



(Figure B-2. Data preprocessing and normalization flowchart illustrating the structured transformation of the MPD dataset from raw source to solver-ready format.)

This diagram captures all major logical branches, including validation, unit conversion, encoding, and workload aggregation. By integrating this standardized workflow, the optimization system guarantees that all inputs used in the MILP formulation (A.1–A.6) are numerically valid, dimensionally consistent, and reproducible for subsequent computational experiments.

B.2.5. Linkage and Reproducibility

This preprocessing component is called automatically during the **Data Preprocessing Layer** of the system architecture (see Appendix B.1). It directly feeds parameter dictionaries to the optimization core (B.4) and ensures synchronization between baseline, MILP, and validation modules.

The output of `clean_mpd()` provides the following mapped variables:

Derived Field	Symbol	Description
Interval_FH	h_i	Task repeat interval in flight hours
TotalMH	MH_i	Total task manhours
ZoneCode	Z_i	Zone or access panel category

SkillCode	S_i	Skill-level category
MenCode	m_i	Manpower category

This standardized mapping enables direct, error-free parameter import by the MILP model, thereby completing the data-to-decision pipeline illustrated in [Figure B-1](#).

B.3. Baseline Heuristic System

This subsection documents the deterministic *block* and *equalized* packaging heuristics used as performance baseline ([§ 4.2](#)). Figure B-3 represents the **traditional maintenance packaging logic** used as *benchmark* (Appendix B.3). It shows how the heuristic assigns tasks sequentially into bins without an optimization solver — purely by procedural rules.

Step	Concept	Relation to Paper
Input	MPD data	Same input as MILP
1	Threshold-based grouping	Encodes block or equalized logic
2	Bin creation	Creates A/C bins manually
3	Sequential assignment	Greedy logic per task
4	Workload & cost calculation	Approximates cost
Output	Baseline cost & schedule	Used in § 4.2 for comparison

The baseline system executes without a solver, assigning tasks sequentially to available check bin based on threshold intervals and predefined grouping logic. Its output structure — task-bin matrix, workload totals, and cost components — mirrors the MILP results, enabling one-to-one comparison.

(Codes)

Figure B-3 illustrates the procedural logic of the heuristic baseline model used for comparative evaluation (§ 4.2).

The algorithm processes normalized MPD data, generates A- and C-check bins, and assigns tasks sequentially according to threshold and interval logic. No mathematical optimization is performed; instead, the algorithm executes deterministic assignment and cost evaluation rules to approximate workload distribution and total maintenance cost. This model serves as the reference benchmark for assessing the efficiency and consistency of the MILP-based optimizer.



Visual styling:

- Rounded rectangles for process boxes
- One diamond for decision ("Interval < A?")
- Use short arrows downward



(Figure B-3 Functional Structure of the Baseline Heuristic Model.)

B.4. MILP Optimization Implementation

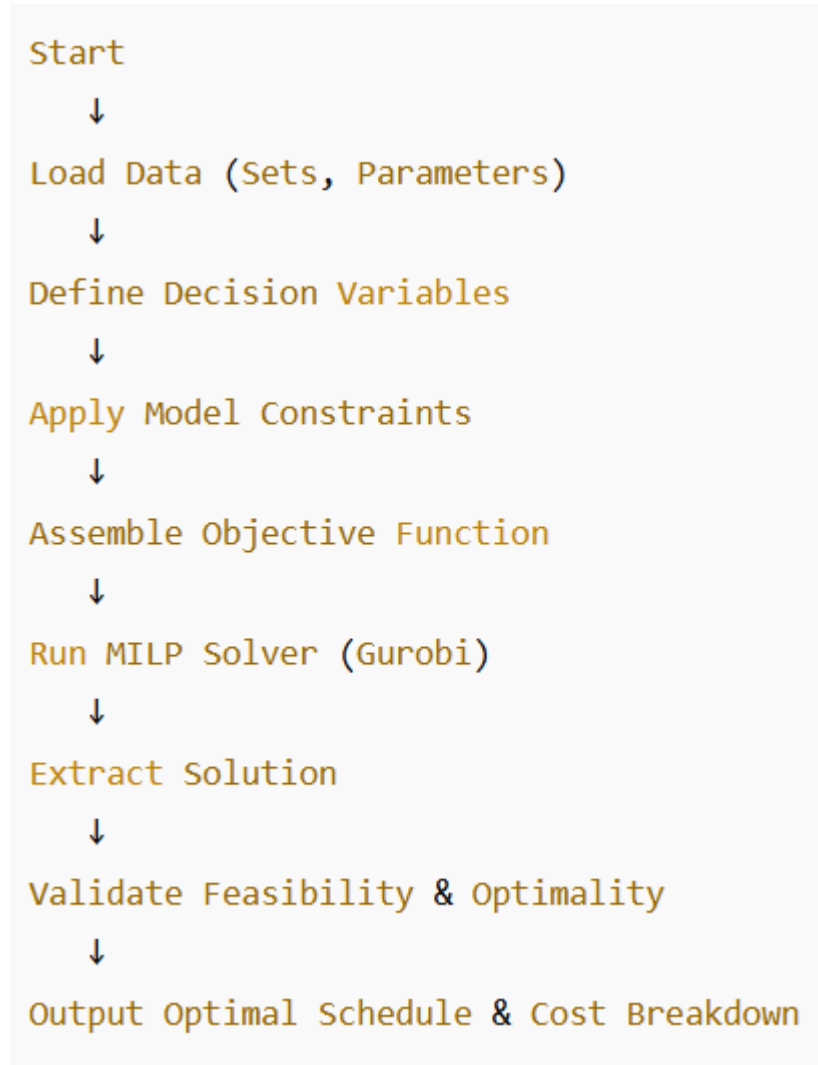
This section provides the Python implementation of the MILP model corresponding to mathematical formulation in Appendix A.

Step	Description	Relation to Math Formulation
1	Load sets, parameters	Appendix A §A.1
2	Define decision variables	Eqs. A8–A10
3	Apply constraints	Eqs. A11–A16
4	Define objective	Eqs. A1–A7
5	Run solver	§ B.3 (Pyomo–Gurobi)
6	Extract results & validate	Eqs. A17–A19, § 3.6
Output	Optimal schedule & cost breakdown	§ 3.5 Results

Each function and constraint block is annotated with its associated equation range for traceability.

(Codes)

This implementation supports both full-scale MPD inputs and reduced-size test cases. Each block of code corresponds directly to the theoretical constructs in Appendix A, ensuring one-to-one mapping between the mathematics and the computational realization.



(Figure B-4 Core Computational Logic of the MILP Optimization Model)

Figure B-4 presents the internal computational sequence of the MILP optimization engine implemented with Pyomo and Gurobi.

The process begins with loading parameter sets and defining binary and continuous decision variables, followed by constraint enforcement and objective function construction. The model is then solved to global optimality (or within the defined MIPGap), after which results are extracted, validated for feasibility, and formatted into interpretable schedules and cost summaries. Each step corresponds directly to the mathematical formulation detailed in Appendix A, ensuring full traceability between equations and code execution.

B.5. Execution Environment and Reproducibility

To ensure replicability of the computational experiments.

Component	Specification
Programming Language	Python 3.11
Optimization Framework	Pyomo 6.7
Solver	Gurobi 11.0
Additional Libraries	pandas 2.2, numpy 1.26, matplotlib 3.9
Hardware	Intel Core i7 @ 3.4 GHz, 32 GB RAM
Operating System	Windows 11 Pro 64-bit
Solver Parameters	MIPGap $\leq 1\%$, TimeLimit = 600 s
Command Line	<code>python run_milp.py --input tasks.csv --delta 100</code>

All experiments were executed using identical parameter files to guarantee consistent results.

A public repository containing the complete scripts, datasets, and solver logs is available upon request to support open-science and reproducibility standards.